

Post-Training Quantization for Large Language Models: A Survey

BAHA RABABAH, Department of Computer Science, University of Manitoba, Canada
YUZHANG SHANG, Department of Computer Science, University of Central Florida, USA
CARSON K. LEUNG, Department of Computer Science, University of Manitoba, Canada
CUNEYT G. AKCORA, Department of Finance, University of Central Florida, USA
MUBARAK SHAH, Department of Computer Science, University of Central Florida, USA

Post-training quantization (PTQ) reduces the memory and compute costs of large language model (LLM) inference by converting a pretrained model from floating-point weights and activations to lower-precision representations after training. Recent studies have shown that LLMs can be compressed to low-bit representations (e.g., 4-bit and 2-bit) with minimal performance degradation. As a result, PTQ has become a standard tool for the efficient deployment of large language models. However, the rapid development of quantization methods has created a fragmented landscape, where approaches differ in error mitigation strategies, calibration procedures, and hardware assumptions, making systematic comparison difficult.

This survey organizes PTQ for LLMs around a single question: How does each method control quantization error? We group existing approaches into four families: compensation-based methods correct errors after quantization, often using Hessian information or low-rank residual terms; rotation-based methods apply orthogonal transforms to redistribute outliers before quantization; salience-based methods identify and protect the weights or channels that matter most; and optimization-based methods learn rounding, clipping, or scaling parameters directly from calibration data.

Beyond this taxonomy, we analyze how individual transformer components, such as attention projections and normalization layers, respond differently to precision loss. We also review calibration strategies, from simple min-max ranges to activation-aware and reconstruction-based approaches, and discuss how the choice of calibration data affects robustness. Finally, we outline open challenges: extreme low-bit quantization, native hardware formats such as MXFP4 and NVFP4, quantizing new architectures like mixture-of-experts and state-space models, and the effect of quantization on long-context and reasoning workloads.

By unifying taxonomy, component-level analysis, and calibration strategy in one framework, this survey aims to give researchers and practitioners a clear, practical reference for understanding and deploying quantized LLMs. We also provide a repository to track developments in this evolving field: <https://github.com/FDataLab/LLMQuantizationSurvey>.

CCS Concepts: • **Computing methodologies** → **Natural language processing; Neural networks**; • **Computer systems organization** → *Neural networks*; • **Software and its engineering** → *Software performance*; • **General and reference** → Metrics.

Authors' Contact Information: Baha Rababah, Department of Computer Science, University of Manitoba, Winnipeg, MB, Canada, rababahb@myumanitoba.ca; Yuzhang Shang, Department of Computer Science, University of Central Florida, Orlando, FL, USA, yuzhang.shang@ucf.edu; Carson K. Leung, Department of Computer Science, University of Manitoba, Winnipeg, MB, Canada, carson.leung@umanitoba.ca; Cuneyt G. Akcora, Department of Finance, University of Central Florida, Orlando, FL, USA, cuneyt.akcora@ucf.edu; Mubarak Shah, Department of Computer Science, University of Central Florida, Orlando, FL, USA, mubarak.shah@ucf.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7341/2026/0-ART000

<https://doi.org/XXXXXXX.XXXXXXX>

Additional Key Words and Phrases: Post-training quantization, large language models (LLMs), model compression, low-bit inference, calibration, hardware-aware deployment

ACM Reference Format:

Baha Rababah, Yuzhang Shang, Carson K. Leung, Cuneyt G. Akcora, and Mubarak Shah. 2026. Post-Training Quantization for Large Language Models: A Survey. *ACM Comput. Surv.* 00, 0, Article 000 (2026), 35 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Post-training quantization (PTQ) reduces the memory and compute cost of large language model (LLM) inference by converting a pretrained model from high-precision weights and activations to lower-precision representations after training. PTQ belongs to the broader area of efficient AI, which aims to make foundation models less expensive to store, move, and execute [55]. These gains, however, come at the cost of quantization error, which can substantially affect model performance. Quantization maps high-precision values to a smaller set of representable levels, and the resulting approximation errors can accumulate across transformer blocks and affect perplexity, generation quality, and downstream task accuracy. Therefore, the central problem in LLM PTQ is to reduce precision while controlling the errors introduced by quantization under practical memory and inference constraints.

Recent methods show that LLMs can often tolerate 8-bit [89] and 4-bit quantization, while 3-bit and 2-bit settings [20, 33] require more careful treatment of outliers [13], sensitive model components and reconstruction error [33, 47]. As quantization becomes more aggressive, the nominal bit-width alone becomes increasingly insufficient for characterizing a method. Existing approaches differ in multiple aspects, such as the tensors they quantize, the granularity at which they assign quantization parameters, the calibration and evaluation datasets they use, the objectives they use to preserve model behavior, the components they retain at higher precision, and the assumptions they make about inference hardware. Consequently, two methods reported at the same bit-width may represent substantially different compression and inference settings, which makes direct comparison difficult.

Previous surveys have examined neural-network quantization [25, 97], LLM compression [108], low-bit LLM systems [26], and transformation-based outlier handling [11]. Table 1 compares these works across PTQ categories, component-aware quantization, calibration strategies, and research challenges. Their scope and emphasis differ substantially. Zhu et al. [108] treated PTQ as one compression technique among several, Yang et al. [97] covered quantization broadly across architectures, and other surveys focus on low-bit systems or specific outlier-handling strategies [11, 26]. As a result, the literature still lacks a unified view that connects the mechanisms used by PTQ methods with where quantization error matters in the model and how calibration choices are made.

Our survey addresses this gap by using quantization error as the main organizing principle for LLM PTQ. This choice is useful because methods with the same bit-width or quantization target can preserve model behavior through very different mechanisms, including error correction, transformations, selective precision, and learned quantization parameters.

An error-control view therefore provides a common basis for comparing methods that would otherwise appear similar from their nominal precision alone. It also connects the method taxonomy to two related questions: how model

Table 1. Comparison of existing surveys. ✓ indicates comprehensive coverage, P indicates partial coverage, and × indicates no substantial coverage.

Survey	Taxonomy	Categories	Components	Calibration	Challenges
Zhu et al. [108]	✓	×	P	×	P
Yang et al. [97]	✓	×	×	×	P
Czakó et al. [11]	✓	P	P	×	P
Gong et al. [26]	✓	P	P	P	✓
This work	✓	✓	✓	✓	✓

components differ in sensitivity to precision loss and how calibration determines quantization decisions.

We organize PTQ methods into four primary error-control mechanisms [104]. Compensation-based methods [48, 101] repair errors introduced by quantization. Rotation-based methods [53, 76, 78] redistribute quantization difficulty through function-preserving transformations of weights and activations. Saliency-based methods [33, 36, 100] identify weights or components that require greater precision or special treatment. Optimization-based methods [7, 72, 87] learn rounding, clipping or equivalent transformations from calibration signals.

Beyond this taxonomy, we examine PTQ from two additional perspectives that are often treated separately. We map the sensitivity of major transformer components to precision reduction and study how calibration strategies estimate quantization parameters and preserve model behavior. We then identify open problems in emerging directions such as extreme low-bit quantization, emerging model architectures, new inference workloads, hardware-supported numeric formats, and standardized evaluation. Together, these perspectives provide a framework for comparing PTQ methods by how they control error, where that error matters in the model, and how the required quantization decisions are obtained.

We summarize our *main contributions* as follows:

- We outline core mathematical principles and design dimensions (e.g., granularity) governing PTQ model accuracy and efficiency.
- We categorize state-of-the-art PTQ methods across four primary families compensation, optimization, saliency, and rotation for weight-only and weight-activation settings.
- We systematically analyze how individual transformer components react to precision reduction and speed trade-offs.
- We identify and systematize emerging calibration strategies, and analyze their role in shaping the accuracy efficiency trade-offs in transformer-based LLMs.
- We distill actionable deployment guidelines, synthesizing method selection, calibration, and hardware-aware optimizations for LLMs.

Article Organization

The article provides a systematic progression from the theoretical foundations of quantization to the practical challenges of large-scale deployment. The sections are organized as follows:

- **Section 2: Background.** Establishes core PTQ mathematical notation, quantization schemes (uniform vs. non-uniform), strategies (weight-only vs. weight-activation), and granularities.
- **Section 3: PTQ Methods as Error-Control Mechanisms.** Taxonomizes state-of-the-art PTQ methods into four primary error-mitigation families: compensation, rotation, saliency, and optimization-based.
- **Section 4: Sensitivity of Transformer Components to Quantization.** Analyzing component-specific quantization strategies across transformer architectures (e.g., KV cache, attention, Layer-Norm, embeddings).
- **Section 5: Calibration Strategies.** Details calibration techniques used to determine quantization parameters, including Min-Max, percentile, KL-divergence, MSE, and zero-shot methods.
- **Section 6: Research Challenges and Future Directions.** Highlights key implementation challenges and emerging research directions in PTQ.

2 Background

Let $\mathcal{M}_\theta$ denote a pretrained decoder-only transformer with parameters $\Theta = \{\Theta_\ell\}_{\ell=1}^L$, where Θ_ℓ denotes the set of weight tensors in transformer block ℓ . We use a Llama-style decoder block as the running example. For hidden size d , query-head count H_q , key-value-head count H_{kv} , and head

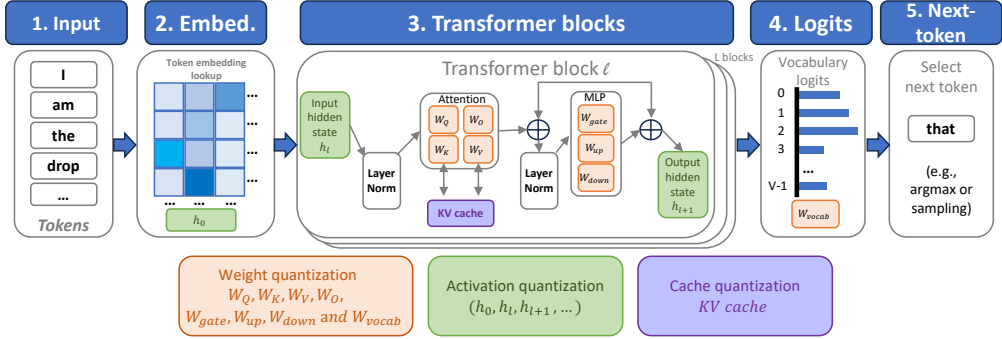


Fig. 1. LLM inference pipeline under quantization. Input tokens are converted to embeddings and processed through repeated transformer blocks. Each block computes query, key, value, and output projections, while the KV cache stores key and value states for decoding. The MLP updates the hidden state before passing it to the next block. Quantization can be applied to weights, activations, and KV-cache states, affecting memory, latency, and output fidelity before logits generate the next token.

dimension d_h , define $d_q = H_q d_h$ and $d_{kv} = H_{kv} d_h$. For row-stacked hidden states $X^{(\ell)} \in \mathbb{R}^{T \times d}$, where T is the sequence length, the attention projections in block ℓ are written as $W_Q^{(\ell)} \in \mathbb{R}^{d \times d_q}$, $W_K^{(\ell)} \in \mathbb{R}^{d \times d_{kv}}$, $W_V^{(\ell)} \in \mathbb{R}^{d \times d_{kv}}$, and $W_O^{(\ell)} \in \mathbb{R}^{d_q \times d}$. The self-attention sublayer forms $Q^{(\ell)} = X^{(\ell)} W_Q^{(\ell)}$, $K^{(\ell)} = X^{(\ell)} W_K^{(\ell)}$, and $V^{(\ell)} = X^{(\ell)} W_V^{(\ell)}$. After head reshaping, attention scores are computed head-wise from query and key vectors, values are aggregated according to the resulting attention weights, and the concatenated attention output is projected back to the residual dimension through $W_O^{(\ell)}$. Under grouped-query attention, $H_{kv} < H_q$, so the key and value projections have a smaller output dimension than the query projection and are shared across groups of query heads. For a generic linear layer with row-stacked input activations X and weight matrix W , the output is $Y = XW$.

PTQ maps floating-point models to low-precision representations while preserving accuracy and deployment efficiency. For example, Llama model weights are commonly stored in 16-bit floating-point formats, such as FP16 or BF16, and PTQ reduces each scalar to lower bit-widths, typically between 2 and 8 bits (Table 2). FP16 and BF16 both use 16 bits per scalar, but allocate these bits differently: FP16 uses 1 sign bit, 5 exponent bits, and 10 mantissa bits, whereas BF16 uses 1 sign bit, 8 exponent bits, and 7 mantissa bits. The quantization scheme determines how values are discretized, which model components are quantized, and the granularity at which quantization parameters are shared.

LLM inference has two phases with different computational profiles: prefill processes the full prompt and builds the KV cache, while decode generates one token at a time by reusing that cache. During prefill, the model reads all input tokens at once, computes their hidden states, and stores their K and V vectors in the KV cache for every transformer block. During decode (the generation phase), the model produces one new token at a time. For each new token, it computes that token's Q, K, and V, reuses cached K and V from earlier tokens, appends the new K and V to the cache, and samples the next token.

Let C denote a finite set of quantization configurations. Each configuration $c \in C$ specifies a bit-width b_c and a quantization method m_c . For a pretrained parameter set Θ , PTQ applies the parameter flow

$$\Theta \xrightarrow{\text{quan}_{b_c}^{m_c}} \Theta_q^{(c)} \xrightarrow{\text{dequan}_{b_c}^{m_c}} \tilde{\Theta}^{(c)}, \quad (1)$$

where $\text{quan}_{b_c}^{m_c}$ maps full-precision parameters to their quantized representation and $\text{dequan}_{b_c}^{m_c}$ maps the quantized representation to the dequantized values used in inference. Thus, $\Theta_q^{(c)} = \text{quan}_{b_c}^{m_c}(\Theta)$ and $\tilde{\Theta}^{(c)} = \text{dequan}_{b_c}^{m_c}(\Theta_q^{(c)})$. The resulting quantized model is denoted by $\mathcal{M}_{\theta_q^{(c)}}$, with inference carried out using dequantized model $\tilde{\Theta}^{(c)}$. For a linear projection $W_A^{(\ell)} \in \Theta_\ell$, where $A \in \{Q, K, V, O, \text{gate, up, down}\}$, we write

$$\mathbf{W}_{A,q}^{(\ell,c)} = \text{quan}_{b_c}^{m_c}(\mathbf{W}_A^{(\ell)}), \quad \tilde{\mathbf{W}}_A^{(\ell,c)} = \text{dequan}_{b_c}^{m_c}(\mathbf{W}_{A,q}^{(\ell,c)}). \quad (2)$$

The difference between a full-precision value and its dequantized value is the quantization error. For a projection matrix A , we write the error as

$$\mathbf{E}_A^{(\ell,c)} = \mathbf{W}_A^{(\ell)} - \tilde{\mathbf{W}}_A^{(\ell,c)}. \quad (3)$$

For row-stacked input activations X , this weight error induces the projection-output error

$$\Delta Y_A^{(\ell,c)} = X\mathbf{W}_A^{(\ell)} - X\tilde{\mathbf{W}}_A^{(\ell,c)} = X\mathbf{E}_A^{(\ell,c)}. \quad (4)$$

When a method first forms a low-bit approximation $\tilde{\mathbf{W}}_A$ and then applies a correction or refinement C , producing $\tilde{\mathbf{W}}_A + C$, we define the residual quantization error as the error remaining after the correction: $E_{\text{res}} = \mathbf{W}_A - (\tilde{\mathbf{W}}_A + C)$. In figures, we use “residual” as a short label.

Notation convention

When the projection, block, and quantization configuration are clear from context, we suppress the indices A , ℓ , and c . We write $\mathbf{W}$ for $\mathbf{W}_A^{(\ell)}$, $\mathbf{W}_q$ for the stored quantized representation, $\tilde{\mathbf{W}}$ for the dequantized low-bit matrix used in computation, and X for the row-stacked input activations. Under this simplified notation, the full-precision and quantized linear layer outputs are $X\mathbf{W}$ and $X\tilde{\mathbf{W}}$, respectively. We use the common shorthand WmA_n to denote quantization with m -bit weights and n -bit activations. For example, $W4A8$ denotes 4-bit weights and 8-bit activations.

Having covered the core functions, we now review the main principles underlying PTQ, focusing on three key dimensions: (i) the distribution of quantization levels (uniform vs. non-uniform), (ii) the target of quantization (weight, cache and activation as shown in Fig. 1), and (iii) the granularity at which quantization parameters are computed and applied.

PTQ converts floating-point weights, cached values and activations into low-bit representations. A fundamental design choice is whether the allowed quantized values are evenly spaced across the numeric range or placed unevenly so that some regions have finer resolution than others [25]. In the literature, evenly spaced quantization values are referred to as uniform quantization, while unevenly spaced values are referred to as non-uniform quantization.

Uniform quantization. Given a bit-width b , uniform quantization maps each floating-point entry of a matrix $\mathbf{W}$ to one of 2^b equally spaced discrete levels. Let $\alpha = \min(\mathbf{W})$ and $\beta = \max(\mathbf{W})$ denote the dynamic range. The scale is $\Delta = (\beta - \alpha)/(2^b - 1)$, and the zero-point is $Z = \text{round}(-\alpha/\Delta)$. For each entry $\mathbf{W}_{ij}$, the stored quantized value ($\mathbf{W}_{ij,q}$) and its dequantized inference value ($\tilde{\mathbf{W}}_{ij}$) are

$$\begin{aligned} \mathbf{W}_{ij,q} &= \text{clamp}\left(\text{round}\left(\frac{\mathbf{W}_{ij}}{\Delta} + Z\right), 0, 2^b - 1\right), \\ \tilde{\mathbf{W}}_{ij} &= \Delta(\mathbf{W}_{ij,q} - Z). \end{aligned} \quad (5)$$

Uniform quantization is computationally efficient and directly supported by integer arithmetic hardware. However, it is highly sensitive to outliers; extreme values in $\mathbf{W}$ can over-expand the range

$[\alpha, \beta]$, leading to a large Δ that collapses the resolution of the remaining values—a phenomenon frequently observed in the heavy-tailed activations throughout transformer blocks.

Non-uniform quantization. Non-uniform quantization maps floating-point values to reconstruction levels that are unequally spaced across the dynamic range. Given a bit-width b , each entry of a weight matrix $\mathbf{W}$ is assigned to one of 2^b codebook entries, with more levels placed in high-density regions of the tensor distribution. This removes the constant step-size constraint of uniform quantization and can better fit the bell-shaped or heavy-tailed distributions often observed in LLM weights and activations [25].

Let $\mathcal{B} = \{r_1, \dots, r_{2^b}\}$ denote the set of non-uniform reconstruction levels. For each entry $\mathbf{W}_{ij}$, the stored quantized representation is the index of the nearest reconstruction level, and the dequantized inference value is the corresponding codebook value:

$$\begin{aligned} \mathbf{W}_{ij,q} &= \arg \min_{k \in \{1, \dots, 2^b\}} |\mathbf{W}_{ij} - r_k|, \\ \tilde{\mathbf{W}}_{ij} &= r_{\mathbf{W}_{ij,q}}. \end{aligned} \quad (6)$$

Here, $\mathbf{W}_{ij,q}$ is the stored codebook index, while $\tilde{\mathbf{W}}_{ij}$ is the dequantized value used in computation.

Quantization scope. Weight-only PTQ quantizes fixed model weights to low-bit formats while keeping activations in FP16 or BF16 during inference [15, 20]. It reduces model storage and memory movement, but latency improves only when inference kernels use the low-bit weights directly rather than first converting them back to FP16 or BF16.

Weight-and-activation PTQ quantizes both weights and runtime activations, enabling low-precision matrix multiplications and larger end-to-end speedups [89]. Because activations depend on the input tokens, W+A quantization requires calibration to estimate activation ranges and handle outliers or token-dependent variation [37].

KV-cache PTQ quantizes the cached keys and values used during autoregressive decoding. It reduces the memory cost of long-context inference, but cache errors are reused at later decoding steps and can distort attention scores or the information retrieved from previous tokens.

Table 2 shows that the main PTQ targets scale differently across Llama model sizes. Moving from 8B to 405B, full-model parameters

grow by about 50×, counted activations by about 15×, and the stored KV cache by about 4× for one 8192-token sequence. These different scaling rates motivate separate quantization strategies for weights, activations, and the KV cache.

Quantization granularity. Quantization granularity specifies which tensor values share the same quantization parameters: those are typically a scale factor and zero-point. Per-tensor quantization uses one shared set of parameters for the whole tensor, making it simple and efficient but sensitive to outliers. Per-channel and per-group quantization use more localized parameters, improving accuracy for uneven or heavy-tailed value distributions at the cost of extra metadata and more complex scaling during dequantization.

A key term in PTQ that will appear throughout this manuscript is on “structured” access. We use structured to mean that selected terms are organized by channels, fixed-size groups, *etc.*, and do

Table 2. Quantization targets and scalar counts for two example Llama models [56, 57].

Component	Llama3 - 8B	Llama3.1 - 405B
<i>Weights</i>		
Full model	8,030,261,248	405,853,388,800
Transformer linear layers	6,979,321,856	401,646,551,040
<i>Activations</i>		
Prefill	7,012,876,288	105,830,678,528
One decode token	856,064	12,918,784
KV cache	536,870,912	2,113,929,216

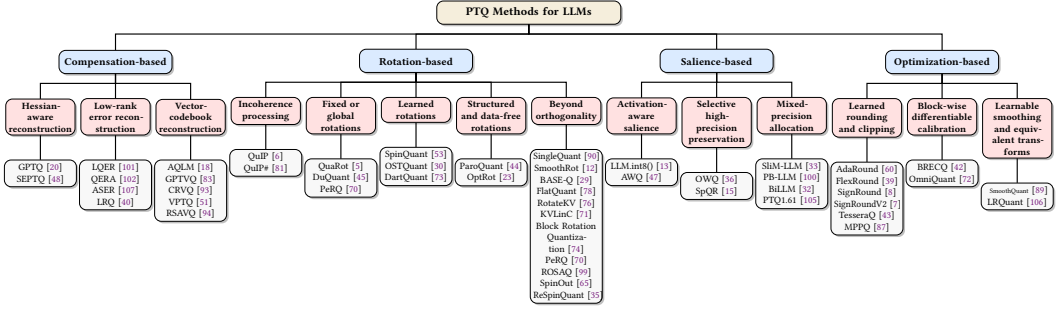


Fig. 2. Taxonomy of PTQ methods for LLMs organized by error-control mechanism.

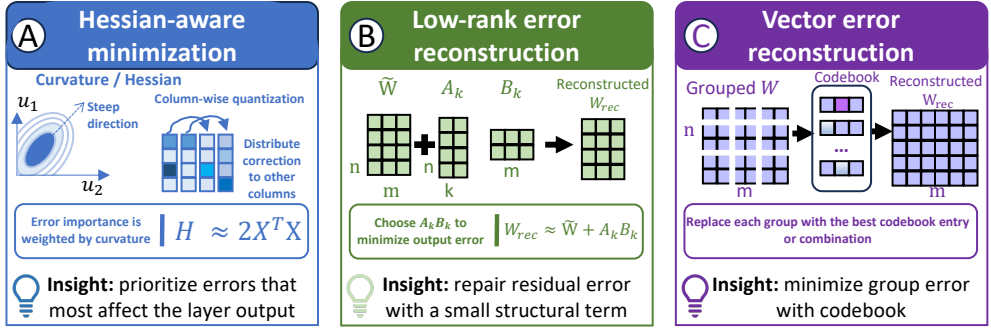


Fig. 3. Compensation-based PTQ methods. Output-error $XW - X\tilde{W}$ is minimized on calibration activations. (A) Hessian-aware methods compensate rounding error during column-wise quantization. The contours show a local quadratic approximation to output or loss change under a quantization perturbation. The directions u_1 and u_2 are Hessian eigen-directions. Perturbations along high-curvature directions are more harmful than perturbations along low-curvature directions. (B) Low-rank reconstruction methods add a compact correction term $A_k B_k$. (C) VQ-based methods reconstruct groups of weights from codebook entries. All three families aim to preserve projection outputs rather than merely minimize entry-wise weight distortion.

not consist of arbitrary individual entries. With unstructured selections, the deployed model must store index data that records which entries are quantized differently and may require irregular memory access or separate high-precision computation at inference time. For this reason, many PTQ methods prefer structure when possible.

3 PTQ Methods as Error-Control Mechanisms

We adopt a four-category taxonomy [104] of compensation-based, rotation-based, saliency-based, and optimization-based methods, and use it as an organizing lens for recent LLM quantization work. The categories are not mutually exclusive: many PTQ methods combine distribution reshaping, calibration-based parameter selection, selective precision, and residual error correction. Therefore, we assign each method to the family that best reflects its primary error-control mechanism. *Compensation-based* methods repair quantization errors, *rotation-based* methods redistribute outliers or sensitive directions before quantization, *saliency-based* methods protect the most influential weights or channels, and *optimization-based* methods learn rounding, clipping, scaling, or related transformations from calibration data. This taxonomy enables comparison by error type, tensors involved, and resulting deployment cost. Figure 2 summarizes the PTQ landscape.

3.1 Compensation-Based Quantization Methods

Compensation-based PTQ methods reduce the accuracy loss caused by low-bit quantization through explicit error correction. They use auxiliary parameters, residual corrections, scaling factors, or second-order optimization to compensate for quantization errors. These methods are especially important for LLMs, where errors in sensitive projections can propagate throughout the transformer. Compensation can be applied at channel, group, vector, or projection level. Existing methods are broadly categorized into Hessian-aware methods, which reduce rounding error during layer-wise quantization; low-rank error-reconstruction methods, which add compact correction terms before or after quantization; and VQ-based methods, which reconstruct groups of weights using learned codebooks.

3.1.1 Hessian-aware reconstruction methods. A representative method in this category is GPTQ [20], which quantizes transformer linear projections by preserving their outputs on a small calibration set. For a projection $\mathbf{W}$, let $\mathbf{X}$ denote the row-stacked calibration activations entering that projection. The goal is to keep the dequantized low-bit output $\mathbf{X}\tilde{\mathbf{W}}$ close to the full-precision output $\mathbf{X}\mathbf{W}$. This gives the linear layer-wise reconstruction objective

$$\|\mathbf{X}\mathbf{W} - \mathbf{X}\tilde{\mathbf{W}}\|_F^2. \quad (7)$$

GPTQ uses approximate second-order information to decide which weight errors are most harmful to the projection output. Although GPTQ quantizes weights, the effect of a weight error is measured after the weight matrix is multiplied by the calibration activations. Under the simplified notation, let $\mathbf{E} = \mathbf{W} - \tilde{\mathbf{W}}$ denote the weight quantization error. The induced output error is $\mathbf{X}\mathbf{E}$, so the reconstruction loss depends on

$$\|\mathbf{X}\mathbf{E}\|_F^2 = \text{Tr}(\mathbf{E}^\top \mathbf{X}^\top \mathbf{X} \mathbf{E}). \quad (8)$$

Thus, the sensitivity of a weight error is determined by the input-activation covariance term $\mathbf{X}^\top \mathbf{X}$. Including the factor from the second derivative, GPTQ uses the Hessian proxy

$$\mathbf{H} = 2\mathbf{X}^\top \mathbf{X}. \quad (9)$$

The activations are used only as calibration data to estimate which weight perturbations change the projection output most. In Fig. 3, the directions $\mathbf{u}_1$ and $\mathbf{u}_2$ illustrate two directions encoded by $\mathbf{H}$: an error along a high-curvature direction changes the reconstruction loss more than the same-sized error along a low-curvature direction. GPTQ uses this information to compensate for rounding errors during weight quantization.

After quantizing one part of a weight matrix, it updates the remaining unquantized weights using inverse-Hessian information so that later weights partially offset the output error introduced by the current rounding decision. To make this scalable, GPTQ processes consecutive blocks of weight coordinates instead of repeatedly updating the whole matrix after every single rounding decision. Within each block, weights are quantized sequentially; after the block is processed, accumulated errors are propagated to the remaining weights through a lazy batch update. GPTQ also replaces repeated inverse-Hessian updates with a Cholesky-based reformulation, which improves numerical stability on very large models. These design choices allow GPTQ to quantize LLM weights to 3–4 bits without retraining and to scale to models with hundreds of billions of parameters.

SEPTQ [48] extends the reconstruction framework by changing which weights are quantized. Suppressing projection, block, and configuration indices for readability, let $\tilde{\mathbf{W}}$ denote the dequantized low-bit matrix obtained from $\mathbf{W}$. SEPTQ computes an importance score for each weight and builds a static mask $\mathbf{M}$, where important weights ($\mathbf{M}_{ij} = 1$) remain in full precision and the rest are

quantized. The effective matrix used after selective preservation is

$$\mathbf{W}_{\text{eff}} = \tilde{\mathbf{W}} + \mathbf{M} \odot (\mathbf{W} - \tilde{\mathbf{W}}). \quad (10)$$

When $\mathbf{M}_{ij} = 1$, the effective value is the original full-precision weight $\mathbf{W}_{ij}$; when $\mathbf{M}_{ij} = 0$, it is the dequantized low-bit value ($\tilde{\mathbf{W}}_{ij}$). During sequential quantization, SEPTQ uses inverse-Hessian updates to propagate rounding errors to the remaining unquantized weights as a post-training compensation step. This selective preservation is particularly effective at very low bit-widths. On OPT-66B at about 2-bit quantization, SEPTQ increases perplexity over the full-precision model by only 8.8% on C4, 6.3% on WikiText2, and 11.8% on PTB, whereas GPTQ increases perplexity by more than an order of magnitude on all three datasets.

Key takeaway. Hessian-aware PTQ methods trade simple rounding for better error compensation: they preserve projection outputs much better at 3–4 bits, but their calibration cost, inverse-Hessian operations, and kernel requirements (*i.e.*, specialized low-level matrix multiplication implementation) make them harder to deploy than round-to-nearest quantization.

3.1.2 Low-rank error-reconstruction methods. Another compensation strategy retains the low-bit matrix as the primary representation and adds a compact high-precision correction term to recover part of the full-precision behavior. Given a full-precision weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and its dequantized low-bit version $\tilde{\mathbf{W}}$, these methods form a reconstructed matrix

$$\mathbf{W}_{\text{rec}} = \tilde{\mathbf{W}} + \mathbf{A}_k \mathbf{B}_k, \quad (11)$$

where $\mathbf{A}_k \in \mathbb{R}^{d_{\text{in}} \times k}$, $\mathbf{B}_k \in \mathbb{R}^{k \times d_{\text{out}}}$, and $k \ll \min(d_{\text{in}}, d_{\text{out}})$. Unlike GPTQ, which compensates rounding errors during quantization, low-rank error-reconstruction methods first quantize the weights and then repair the residual quantization error by adding the low-rank term $\mathbf{A}_k \mathbf{B}_k$.

LQER [101] is an early low-rank error-reconstruction method for LLM PTQ. It represents the full-precision weight matrix as a low-precision high-rank matrix plus a high-precision low-rank correction. Instead of directly approximating the quantization error $\mathbf{W} - \tilde{\mathbf{W}}$, LQER first rescales it using activation magnitudes before applying SVD, allowing the low-rank correction to better reduce projection-output error. Unlike methods such as LLM.int8(), which preserve activation outliers in FP16 and require scatter-and-gather memory access, LQER computes $\mathbf{X}\tilde{\mathbf{W}}$ and adds the correction $(\mathbf{X}\mathbf{A}_k)\mathbf{B}_k$, avoiding scattered high-precision lookups. In the W4A8 setting, LQER achieves near-lossless downstream accuracy while using $1.36\times$ fewer FPGA resources than the strongest baseline at the same throughput target.

QERA [102] provides an analytical formulation of low-rank error reconstruction. It shows that minimizing the weight approximation error $\|\mathbf{W} - \tilde{\mathbf{W}} - \mathbf{A}_k \mathbf{B}_k\|_F$ does not necessarily minimize the projection-output error. Instead, QERA derives a closed-form correction $\mathbf{C}_k = \mathbf{A}_k \mathbf{B}_k$ by minimizing the calibration-output discrepancy $\|\mathbf{X}\mathbf{W} - \mathbf{X}(\tilde{\mathbf{W}} + \mathbf{C}_k)\|_F^2$, where $\mathbf{X}$ contains row-stacked calibration activations. Hence LQER is an activation-scaled SVD heuristic, whereas QERA directly optimizes the output-error objective.

ASER [107] combines low-rank error reconstruction with activation smoothing. It models the activation-weight quantization error as a low-rank structure and reconstructs it using LoRA-style matrices [14] while smoothing activation outliers that contribute most to the error. This makes ASER well suited for weight-activation quantization. The method recovers much of the accuracy loss in W4A8 per-channel quantization with minimal overhead. For LLaMA-3.1-8B on WikiText-2, ASER W4A6 reports a relative perplexity increase of 36.97% over the FP16 model.

LRQ [40] uses low-rank structure differently by learning low-rank weight-scaling matrices during block-wise reconstruction rather than adding a low-rank correction to the quantization error. The objective is to preserve intermediate transformer block outputs while avoiding the overfitting associated with learning an independent scaling factor for every weight. This approach provides greater flexibility than channel-wise scaling with far fewer calibration parameters than element-wise scaling. Evaluated under W8A8 and W4A8 quantization, LRQ achieves higher task accuracy than SmoothQuant [89] and FlexRound [39].

Key takeaway. Low-rank error reconstruction treats quantization as a base approximation plus a structured repair term. Its strength is that a small high-precision correction can recover projection-output behavior without retraining the whole model; its weakness is that the correction must remain low-rank and hardware-friendly, otherwise the extra matrices can erode the memory and speed benefits of quantization.

3.1.3 Vector-quantization-based compensation methods. Vector quantization (VQ) compresses weights by replacing short weight vectors with indices into a codebook of representative vectors. During inference, each index retrieves its corresponding vector, so the model stores index matrices and codebooks instead of full-precision weights. Unlike scalar quantization, which quantizes each weight independently, VQ jointly quantizes groups of weights and captures recurring local patterns. This is particularly beneficial in the 2–3 bit regime, where scalar quantization provides too few levels to represent weight distributions accurately. However, VQ introduces decoding overhead because indices must be mapped back to vectors before matrix multiplication. Consequently, VQ methods balance reconstruction quality, codebook size, and codebook lookup latency during inference.

AQLM [18] applies additive vector quantization to LLM weights for extreme compression, targeting 2–3 bits per parameter. Instead of representing each vector by a single codebook entry, AQLM reconstructs a vector as the sum of entries from multiple codebooks. This increases reconstruction capacity without requiring a single very large codebook. AQLM also uses calibration activations so that the quantized weights preserve projection and transformer-block outputs, and it jointly tunes codebook parameters across transformer blocks. Its main strength is the size–perplexity trade-off in the 2-bit regime; its main cost is a more complex decoding path and larger lookup structure than simpler VQ formats.

GPTVQ [83] focuses on hardware-friendly VQ for mobile CPU inference. It treats VQ primarily as a storage format: FP16 weights are split into groups, each group receives a small codebook and an index table, and the indices are decoded to a native type before matrix multiplication. In its reported mobile CPU setting, GPTVQ reduces DRAM footprint by 19% and improves token rate by 10% over a standard INT4 baseline.

CRVQ [93] extends VQ to sub-2-bit quantization by allocating extra codebook capacity only to critical weight channels, whose quantization errors have the greatest impact on linear layer outputs. Channel importance is estimated using a Hessian-based proxy that combines pre-quantization error with calibration activations. After ranking the channels, CRVQ reorders the weight matrix so that critical channels are grouped together. It then applies one shared base codebook to all groups and additional codebooks only to the critical groups. In its $8 + 8 \times 3$ codebook setting, the first 8 denotes an 8-bit base codebook, and 8×3 denotes three additional 8-bit extension codebooks assigned only to critical groups. Because the extension codebooks are used selectively, the average storage remains about 1.06 bits per weight rather than 32 bits. For LLaMA-3.1-8B on WikiText-2, AQLM at 1.01 bits increases perplexity by 891.44% over FP16, whereas CRVQ at 1.07 bits reduces the increase to 368.49%.

VPTQ [51] formulates VQ as a second-order reconstruction problem. It uses curvature information to guide the choice of codebook entries and to reduce the effect of vector-level reconstruction error on the projection output. VPTQ further uses residual vector quantization, where a second codebook quantizes the error left by the first codebook, and outlier handling, where a small fraction of difficult vectors is quantized with a separate configuration. These choices are designed to improve 2-bit weight-only quantization while keeping decoding overhead low. The article reports that, at 2-bit quantization, VPTQ reduces perplexity relative to prior methods and achieves 1.6–1.8× higher inference throughput than AQLM, GPTQ, and GPTVQ on LLaMA-2 7B and 13B models, while showing comparable throughput on the 70B model.

RSAVQ [94] addresses two limitations of VQ: the direction of the vector reconstruction error and the unequal sensitivity of weight channels. It uses Fisher-information-based sensitivity estimates to push quantization error toward directions that have lower effect on the loss, and to allocate more bits to channels that are more sensitive to perturbation. On LLaMA-3 8B at 2 bits, RSAVQ increases WikiText-2 perplexity by 43.16% over the FP16 model and reduces average zero-shot accuracy by 10.11%. Under the same setting, VPTQ increases perplexity by 51.30% over FP16 and reduces average zero-shot accuracy by 12.29%.

Key takeaway. At 2 bits, scalar quantization largely runs out of representation capacity, and VQ becomes the more natural compression model for LLM weights. However, a method must keep codebooks and lookup latency small enough to turn better reconstruction into real inference gains.

3.2 Rotation-based Methods

Rotation-based PTQ improves LLM quantization by applying orthogonal transformations to weights and activations. These computationally invariant rotations redistribute activation outliers across channels, which shrinks the quantization range, flattens the value distribution, and makes low-bit inference more accurate. Existing methods are broadly categorized into incoherence-based rotations that reduce the concentration of outliers, fixed/global rotations, learnable rotations, and advanced layer-wise or closed-form approaches.

Rotation-based PTQ relies on the computational invariance of the transformer’s residual stream. Since RMSNorm is invariant to orthogonal transformations, inserting an orthogonal matrix $\mathbf{Q}$ into the residual stream and absorbing $\mathbf{Q}^\top$ and $\mathbf{Q}$ into adjacent linear layers preserves the full-precision output exactly [5]. For a linear layer, this follows from $\mathbf{Y} = \mathbf{X}\mathbf{W} = (\mathbf{X}\mathbf{Q})(\mathbf{Q}^\top\mathbf{W})$, so a single rotation propagates through an entire transformer block. In practice, $\mathbf{Q}$ redistributes activation outliers across channels, reducing per-channel dynamic range and improving low-bit quantization, while fused rotations incur no inference overhead. Rotations that cannot be fused because of operations such as RoPE or softmax are applied online, typically using Hadamard matrices with fast Walsh–Hadamard transforms. QuaRot and SpinQuant implement four rotations: $\mathbf{R}_1$ on the residual stream and $\mathbf{R}_2$ on the attention value/output path (both offline), and $\mathbf{R}_3$ on post-RoPE queries/keys and $\mathbf{R}_4$ on the down-projection input (both online for KV-cache and activation quantization, respectively) [5, 53].

Two properties characterize rotation-based PTQ. First, computational invariance makes rotation an efficient outlier-suppression technique, which explains its widespread use. Second, orthogonal rotations are limited: because they preserve vector norms, they cannot correct channel-range asymmetry, mean misalignment, or severe attention-sink outliers (*i.e.*, tokens that receive a disproportionate share of attention while carrying little semantic information). Consequently, many recent methods combine rotation with non-orthogonal operations such as scaling, permutation, bias correction, mixed precision, or outlier isolation. These methods can be compared along two

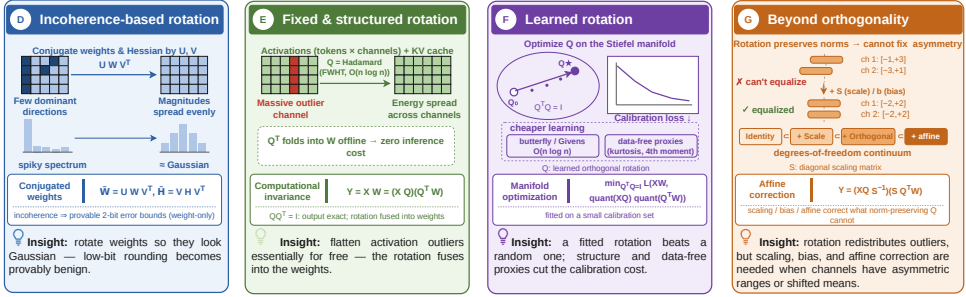


Fig. 4. Rotation-based PTQ methods. All four mechanisms reshape the distributions the quantizer sees without changing the layer function. (D) Incoherence-based rotations conjugate the weights and Hessian by random orthogonal or Hadamard matrices to spread outliers more evenly across dimensions; this reduces the dominance of individual directions and enables provable error bounds for low-bit weight-only quantization. The remaining panels exploit the computational invariance $Y = XW = (XQ)(Q^T W)$ for an orthogonal Q . (E) Fixed and structured rotations extend the transform to activations and the KV cache, spreading outlier energy evenly across channels with data-independent matrices that fold into the weights at no inference cost. (F) Learned rotations instead optimize the orthogonal matrix on a small calibration set, with structured parameterizations and data-free objectives reducing the cost of that optimization. (G) Beyond orthogonality, methods add degrees of freedom outside the orthogonal group (diagonal scaling, bias correction, or full affine transforms) to repair the per-channel range asymmetry and massive outliers that norm-preserving rotations cannot fix.

dimensions: their transformation degrees of freedom (identity $\subset$ diagonal scaling $\subset$ orthogonal rotation $\subset$ affine) and whether the transformations are fused offline or applied online at inference. Figure 4 summarizes the main categories of rotation-based PTQ methods.

3.2.1 Incoherence-based rotations. Rotation was first introduced in LLM PTQ to obtain provable low-bit error bounds rather than to suppress outliers. QuIP [6] conjugates the weight matrix and its Hessian with random orthogonal matrices. The conjugation makes both more incoherent, so adaptive rounding becomes nearly optimal; this enabled the first practical 2-bit weight quantization with theoretical guarantees. QuIP# [81] replaces random rotations with randomized Hadamard transforms for tighter incoherence bounds and efficient $O(n \log n)$ computation, then applies E_8 -lattice vector quantization and brief fine-tuning to reach 2-bit weight-only quantization competitive with earlier 3-bit methods. Both methods follow a rotation-plus-codebook strategy but operate only on weights and the Hessian, leaving activations unchanged.

3.2.2 Fixed and structured rotations. Later work extended rotation from weights to the entire forward pass, including activations and the KV cache, using training-free transforms that can be fused offline. QuaRot [5] applies a global randomized Hadamard rotation to the residual stream (R_1) together with in-block rotations R_2 – R_4 , providing the first full W4A4 pipeline with 4-bit KV-cache quantization. On LLaMA-2-70B, it increases WikiText-2 perplexity by less than 0.5, preserves about 99% of zero-shot accuracy, and achieves up to $2.16\times$ faster prefill. While computationally efficient, its data-independent global rotation has limited expressivity. DuQuant [45] retains the training-free design but replaces global Hadamard rotations with a deterministic, outlier-aware approach. It constructs a block-diagonal rotation by grouping outlier dimensions, applies a zigzag permutation to balance block-wise variance, and finishes with a smoothing rotation. By combining rotation with permutation, DuQuant addresses the large outliers that pure Hadamard rotations cannot eliminate, a design later supported theoretically by PerQ [70].

Table 3. Taxonomy of rotation-based PTQ methods. The primary reading axis is **Transform DOF**, which traces the degrees-of-freedom continuum (identity $\subset$ scaling $\subset$ orthogonal $\subset$ affine) that organizes the discussion. *Placement* indicates where the transform lives at inference (offline/fused vs. online); *Augmentation* lists the degree of freedom added off the orthogonal group (“rotation + X ”). Groups follow the order of discussion.

Method	Transform DOF	How obtained	Placement	Targets	Augmentation	Principle
QuIP [6]	orthogonal	random (Kronecker)	offline (W + Hessian)	W	adaptive rounding	incoherence
OSTQuant [81]	orthogonal	randomized Hadamard	offline	W	E_8 VQ + finetune	incoherence
QuaRot [5]	orthogonal	random Hadamard	R_1/R_2 offline, R_3/R_4 online	W+A+KV	—	partial
DuQuant [45]	block-orthogonal	greedy (outlier-prior)	offline	W+A	permutation (zigzag)	—
SpinQuant [53]	orthogonal	learned R_1/R_2 (Cayley)	R_1/R_2 offline, R_3/R_4 online	W+A+KV	—	—
OSTQuant [30]	orthogonal+scale	learned	offline	W+A+KV	scaling + KL-Top loss	QSUR
KurTail [2]	orthogonal	learned (min. kurtosis)	offline, layer-wise	W+A+KV	—	kurtosis proxy
DartQuant [73]	orthogonal	learned (distrib.-aware, QR)	offline	W+A	—	—
ButterflyQuant [91]	orthogonal (butterfly)	learned Givens rotation, $O(n \log n)$	online (structured)	A (2-bit)	uniformity reg.	—
ParoQuant [44]	orthogonal (pairwise)	learned pairwise Givens	online (structured)	W+A	scaling	—
OptRot [23]	orthogonal	learned, data-free (4th-power W)	offline (fusible)	W (+A in OptRot*)	—	4th-moment proxy
SingleQuant [90]	orthogonal	closed-form Givens rotation (ART+URT)	offline	W+A	alignment + uniformity	partial
SmoothRot [12]	orthogonal	fixed Hadamard	online (+ fused scaling)	W+A	scaling (SmoothQuant)	—
BASE-Q [29]	orthogonal (fixed)	fixed, no learned rot.	offline	W+A	bias + asym. scaling	—
FlatQuant [78]	affine	learned affine (Kronecker)	online (fused kernel)	W+A	scaling (affine $>$ scale)	flatness
RotateKV [76]	orthogonal	adaptive Hadamard + re-order	online (pre-RoPE)	KV	perm. + sink protect	—
KVLinC [71]	orthogonal	fixed Hadamard	online	KV	linear correction	—
BRQ [74]	block-orthogonal	block Hadamard	online (block)	W+A (MXFP4)	format match	format-aware
PeRQ [70]	block-orthogonal	block Hadamard + perm.	online (block)	W+A	permutation	L_1 -mass bound
ROSAQ [99]	PCA/eigen	closed-form PCA	offline	W (salient FP16)	mixed precision	PCA
SpinOut [65]	orthogonal	learned + outlier injection	as SpinQuant	W+A+KV	layer-sensitivity search	—
ReSpinQuant [35]	orthogonal	learned layer-wise, re-folded	offline (subspace residual)	W+A	subspace decomposition	—

3.2.3 Learned rotations. When fixed rotations are insufficient, the orthogonal transform can be learned. SpinQuant [53] optimizes the two fusible rotations R_1 and R_2 on the Stiefel manifold using a small calibration set, while retaining the fixed online Hadamard rotations R_3 and R_4 from QuaRot. Learning these rotations substantially narrows the W4A4 accuracy gap of random rotations but requires dense $O(n^2)$ parameters and backpropagation through the quantized model. OSTQuant [30] jointly learns an orthogonal rotation and diagonal scaling, introducing the Quantization Space Utilization Rate (QSUR) to show that combining both improves quantization efficiency. KurTail [2] instead minimizes the kurtosis of rotated activations layer by layer, reducing memory usage while outperforming QuaRot. DartQuant [73] lowers optimization cost by replacing full backpropagation with distribution-aware calibration constraints and QR-based orthogonalization. Together, these methods buy higher quantization accuracy at the cost of a more involved calibration step.

3.2.4 Structured and data-free rotations. Two strategies reduce the cost of learned rotation. One constrains the transform to a sub-quadratic structured family. ButterflyQuant [91] parameterizes the rotation as a learnable butterfly transform built from continuous Givens-rotation angles; it is orthogonal by construction, uses $O(n \log n)$ compute and only $(n \log n)/2$ parameters, adds a uniformity regularizer on the transformed activations, and reaches 2-bit activations while converging in minutes on a single GPU.

ParoQuant [44] similarly uses independent pairwise Givens rotations together with channel-wise scaling, but is motivated by reasoning LLMs, where quantization error compounds over long chains

of thought; it improves over AWQ by 2.4% on reasoning tasks with under 10% overhead. The other strategy removes calibration data altogether: OptRot [23] learns fusible rotations by minimizing a data-free proxy, the elementwise fourth power of the rotated weights, which is a kurtosis-like objective defined on weights rather than activations (a counterpart to KurTail [2]), and outperforms Hadamard, SpinQuant, and OSTQuant for weight quantization; a data-dependent variant, OptRot⁺, adds activation-covariance information for W4A8. These methods recover most of SpinQuant’s accuracy without its dense $O(n^2)$ optimization.

3.2.5 Beyond orthogonality: scaling, bias, and affine transforms. Because norm preservation lets a rotation equalize variance across channels but not per-channel range asymmetry, channel-mean offsets, or massive outliers, the strongest methods add degrees of freedom outside the orthogonal group. SingleQuant [90] constructs two closed form Givens transforms: an alignment rotation that analytically smooths massive outliers and a uniformity rotation that reshapes the remaining distribution toward uniform. It avoids *straight-through Stiefel optimization* and reports transform construction roughly three orders of magnitude faster than optimization-based rotation. SmoothRot [12] composes the two operations directly, applying SmoothQuant-style per-channel scaling first to tame massive outliers and hadamard rotation second to spread the residual, without added inference latency. BASE-Q [29] argues that for channel-mean misalignment and asymmetric ranges a learned rotation is the wrong tool, and replaces it with bias correction and asymmetric scaling on top of a cheap fixed rotation. FlatQuant [78] generalizes the transform further, learning a per-linear-layer affine (invertible-linear) transform that is strictly more expressive than any orthogonal map in order to maximize the flatness of the weight and activation distributions; it controls cost through a Kronecker decomposition into two small matrices and fuses the transform and quantization into a single kernel, improving over SpinQuant by 7.5% at W4A4 on LLaMA-3-70B while reducing transform overhead. The QSUR derivation of OSTQuant [30] gives a formal statement of the same observation, that orthogonal-plus-scaling dominates rotation alone, while ROSAQ (discussed below) instead uses rotation to decide which channels deserve extra precision. **Specialized settings.** Rotation interacts with other components of the model, and RoPE, the KV cache, and the emerging FP4 hardware formats each complicate a naive rotation. For the KV cache, RotateKV [76] achieves robust 2-bit KV quantization by adapting the rotation to per-channel outlier patterns through channel reordering (while remaining Fast Walsh–Hadamard Transform compatible), rotating before RoPE with grouped heads to avoid RoPE’s interference with the rotation, and protecting attention-sink tokens; it reports under 0.3 perplexity degradation at 2-bit KV on LLaMA-2-13B. KVLinC [71] uses a lighter recipe, hadamard rotating the values and adding small linear-correction adapters to compensate for key-quantization error. On the hardware side, Block Rotation Quantization [74] observes that global hadamard rotation can degrade accuracy under MXFP4’s per-block power-of-two scaling, because uniform energy-spreading wastes the block’s dynamic range, and matches the rotation block size to the MXFP4 scaling block to address this. PerQ [70] provides a corresponding analysis for block rotation in general, proving non-asymptotically that post-rotation outliers are minimized when the pre-rotation L_1 mass is balanced across blocks, and adds a greedy mass-diffusion permutation that recovers up to 90% of full-rotation accuracy, complementing DuQuant’s empirical use of rotation and permutation. ROSAQ [99] connects rotation to the salience family: it uses a closed-form PCA rotation so that the salient directions coincide with the top principal directions, then keeps those few directions in FP16 while quantizing the rest, which makes it a rotation–salience hybrid rather than a learned rotation. FrameQuant, which uses *overcomplete fusion frames* rather than square orthogonal matrices, is a related transform-domain quantizer but is not rotation-based [1].

Robustness and offline–online trade-offs. Recent work focuses on stabilizing learned rotations and on reconciling expressivity with fusibility. SpinOut [65] stabilizes learned rotations by injecting artificial outliers during rotation training into a searched subset of outlier-sensitive layers, scored by kurtosis and performance metrics. This lowers variance and cuts the calibration requirement by 75% relative to SpinQuant. ReSpinQuant [35] addresses the offline–online trade-off. Layer-wise rotations are the most accurate but normally run online, so it refolds them into the weights offline and resolves the resulting basis mismatch between adjacent layers with a subspace residual rotation approximation. The result matches expensive layer-wise methods at W4A4 and W3A3 with little runtime overhead. ReSpinQuant [35] and FlatQuant [78] approach this trade-off from opposite directions, one refolding layer-wise transforms into the offline regime and the other fusing an expressive online transform into a single fast kernel.

Key takeaway. Two themes summarize this family of methods. The first is computational invariance of the residual stream, which turns outlier suppression into a transform that folds into the weights at no inference cost; this is why rotation is so widely used as a front-end for low-bit LLM inference. The second is the family’s main limitation: orthogonal maps preserve norm, so they cannot fix per-channel asymmetry, mean offsets, or massive outliers. Most competitive methods therefore add operations outside the orthogonal group, either moving along the degrees-of-freedom continuum from rotation toward affine (FlatQuant, OSTQuant, SmoothRot, BASE-Q, ParoQuant) or adding permutation (DuQuant, PerQ, RotateKV), bias (BASE-Q), mixed precision (ROSAQ), or codebooks (QuIP#). Several open problems remain: characterizing which layer-wise transforms admit zero-overhead offline refolding; a unified account of rotation-plus-auxiliary-operation designs beyond PerQ’s L_1 -mass bound; format-aware rotations for MXFP4 and NVFP4 hardware; rotation under newer attention mechanisms (Grouped Query Attention, Multi-head Latent Attention) and Mixture-of-Experts (MoE) routing; error accumulation in reasoning models; reliable W2A2 quantization; and a controlled benchmark that addresses the evaluation drift caused by inconsistent calibration and KV-cache protocols.

3.3 Saliency-Based Quantization Methods

Saliency-based PTQ starts from the observation that quantization error is not equally harmful across all values of a transformer. For a linear layer with weights $\mathbf{W}$ and calibration activations $\mathbf{X}$, an error term $\Delta\mathbf{W}$ affects the layer through $\mathbf{X}\Delta\mathbf{W}$. A weight, channel, column, or group is therefore salient when perturbing it causes a large change in the projection output or downstream loss. Existing methods estimate this importance from activation magnitudes, weight magnitudes, Hessian or inverse-Hessian scores, output reconstruction error, and distributional outlier statistics. After estimating saliency, they quantize selectively through scaling, high-precision side paths, mixed bit-width assignment, or binary masks. The primary decision in saliency-based PTQ is which parts of the model are most sensitive to quantization and therefore require greater precision.

Figure 5 summarizes this view through three mechanisms. Activation-aware methods protect channels whose weights are amplified by typical inputs. Selective preservation isolates rare outlier features or weights and keep them in higher precision. Structured allocation methods assign bit-widths, masks, or scaling groups according to saliency, especially in sub-2-bit and binary regimes.

3.3.1 Activation-aware saliency and outlier channels. The earliest LLM-specific form of saliency-aware quantization appears in LLM.int8() [13]. The method targets 8-bit matrix multiplication at scale, where a small number of emergent feature dimensions can dominate transformer behavior. It combines vector-wise 8-bit quantization for the dense part of the matrix multiplication with a 16-bit mixed-precision decomposition for outlier dimensions. In this sense, LLM.int8() treats

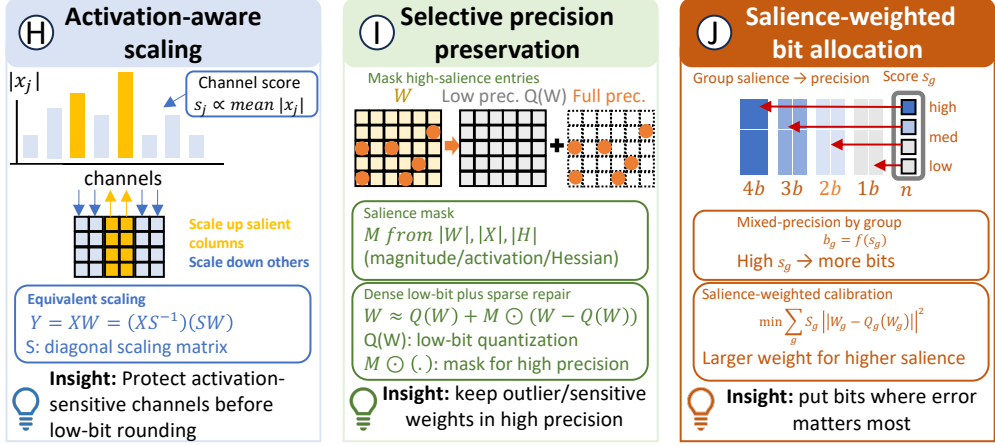


Fig. 5. Saliency-based PTQ methods. (H) Activation-aware scaling methods use calibration activations to identify sensitive channels and apply equivalent transformations that reduce quantization error without changing the layer function. (I) Selective-preservation methods use a binary saliency mask M to keep selected weights in higher precision, while the remaining parameters use the dequantized low-bit matrix $\tilde{W}$. (J) Saliency-weighted bit-allocation methods estimate group importance and assign mixed precision accordingly.

the outlier feature subspace as high saliency and prevents it from being rounded into the same 8-bit representation as ordinary channels. Its main contribution is the principle that a small set of activation-driven outlier dimensions can determine whether quantization preserves model quality.

AWQ [47] gives the standard activation-aware formulation for low-bit weight-only PTQ (which was discussed in Section 3.1). AWQ observes that protecting a very small fraction of salient weights can sharply reduce quantization error, but it identifies salient weight channels using activation statistics, not weight magnitude alone. For a weight channel that receives large calibration activations, a small weight perturbation can produce a large output error. AWQ avoids irregular mixed precision by applying an equivalent scaling transformation: salient weight channels are scaled up before quantization, and the corresponding activations are inversely scaled so that the full-precision function is preserved. The scaling factors are selected through lightweight calibration by minimizing projection-output reconstruction error. This makes AWQ easier to deploy because the scaling learned during calibration is absorbed into the stored low-bit weight matrix. In the final matrix W_q , every entry is quantized, and fixed groups of weights share their quantization scale. During inference, the model does not need to store selected outlier weights in FP16 or compute them with a separate high-precision multiplication.

3.3.2 Selective high-precision preservation. A second line of work keeps a small sensitive subset in higher precision and quantizes the dense remainder aggressively. OWQ [36] identifies weak columns, i.e., structured weight columns that are especially sensitive to quantization. These columns are stored in higher precision, while the remaining weights are quantized to low-bit values. The method keeps the preserved subset structured enough to avoid fully unstructured sparse storage. OWQ also introduces Weak Column Tuning, a small adaptation step that improves task performance after quantization with limited memory overhead.

SpQR [15] applies a related idea at the level of outlier weights. It identifies weights that cause unusually large quantization error, stores them in a sparse higher-precision representation, and quantizes the remaining dense weights with group-wise scales. The result is a sparse-quantized

format that can reach near-lossless compression at 3–4 bits per parameter. SpQR shows that the error distribution of LLM weights is sufficiently concentrated that isolating a small outlier set can recover much of the full-precision behavior. Its systems cost is the need to encode, store, and decode the high-precision sparse component during inference.

These selective-preservation methods make a different accuracy efficiency trade-off from AWQ. AWQ protects salient channels through dense equivalent scaling, so it stays close to ordinary low-bit kernels. OWQ and SpQR spend explicit high precision on selected columns or entries, which can improve accuracy but introduce extra metadata, mixed-precision paths, and possible memory-access irregularity.

3.3.3 Structured mixed precision and binary salience. SliM-LLM [33] moves salience protection from individual exceptions to group-level bit allocation. It uses the structured distribution of weight salience to assign different bit-widths to groups within a layer. Its *salience-determined bit allocation* selects the precision of each group according to global salience structure, while its *salience weighted quantizer calibration* adjusts quantization parameters using element-level salience inside each group. This design preserves important weights without introducing arbitrary sparse exceptions, so it is better aligned with dense integer inference than element-wise mixed precision.

PB-LLM [100] extends salience protection to the binary regime. The method first selects a small subset of salient weights, using magnitude-based or Hessian-based criteria, and stores those weights in higher precision. The remaining weights are binarized. In its PTQ variant, PB-LLM uses Hessian-guided reconstruction to recover the binarized weight matrix, following the same intuition as GPTQ that quantization error should be measured by its effect on projection outputs. In its training-aware variant, salient weights are frozen and the residual binarized weights receive scaling factors derived to reduce quantization error. For our taxonomy, PB-LLM is most relevant as an extreme example of selective precision spending: binarization becomes viable only after high-salience weights are protected.

BiLLM [32] further develops 1-bit PTQ by separating salient and non-salient weights according to weight-distribution structure. It first selects salient weights in a structured way, then applies a binary residual approximation to reduce the error caused by compressing those weights. For the remaining non-salient weights, it uses the bell-shaped distribution of pretrained weights to guide splitting and binarization. BiLLM therefore combines two salience decisions: which weights require special treatment, and how the lower-salience residual population should be partitioned before binarization.

PTQ1.61 [105] pushes the same idea into sub-2-bit PTQ with a more structured mask but with a salience selection that is cheaper to store and easier to execute. It uses input activations to construct a one-dimensional structured mask that selects salient channels for 4-bit quantization, adding negligible mask overhead per weight. Non-salient channels are then binarized with block-wise scale optimization that accounts for row-wise correlations and angular bias. PTQ1.61 also introduces quantization preprocessing, where the pretrained weight distribution is transformed before low-bit quantization to reduce the difficulty of per-channel binarization.

Key takeaway. Salience-based PTQ protects a small subset of weights, channels, or groups and compresses the remainder more aggressively. Its effectiveness stems from the fact that quantization error is highly concentrated: activation outliers, weak columns, sparse outlier weights, and sensitive groups often account for a disproportionate share of model degradation. The main challenge is identifying these salient structures and preserving them without introducing excessive storage or inference overhead. Early methods often retain isolated weights, columns, or channels in higher precision, creating irregular computation paths. More recent approaches

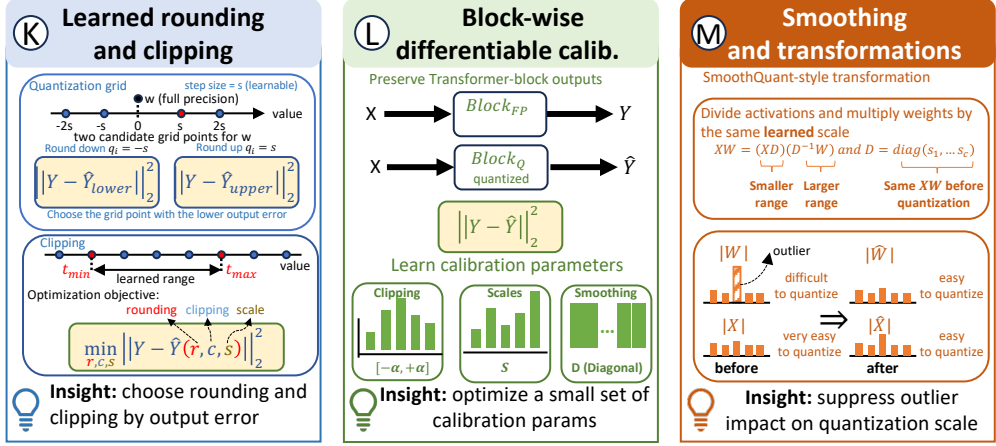


Fig. 6. Optimization-based PTQ methods. (K) Learned rounding and clipping methods use calibration data to set rounding choices and clipping thresholds by minimizing output reconstruction error. (L) Block-wise differentiable calibration methods preserve the outputs of full transformer blocks by optimizing a small set of quantization parameters, such as clipping thresholds, scales, and transformations. (M) Smoothing and transformation methods reduce activation outliers through equivalent scaling operations that preserve the full-precision product while making activations easier to quantize. Despite their differences, all three families treat quantization as a calibration-time optimization problem whose objective is to minimize the discrepancy between full-precision and quantized outputs.

allocate precision at the channel or group level, which preserves hardware efficiency because quantization parameters and bit-width assignments can be applied to contiguous blocks of weights during matrix multiplication.

3.4 Optimization-based Methods

Optimization-based PTQ methods, as shown in Fig. 6, treat quantization choices as variables estimated from calibration data. The calibration data is a small set of unlabeled input sequences used to measure how a layer or transformer block behaves before and after quantization. The optimized variables include rounding decisions, clipping ranges, integer-to-floating scales, smoothing factors, and equivalent transformations.

This family is related to compensation-based PTQ, but the mechanism is different. Compensation-based methods repair the error after a low-bit approximation has been chosen. Optimization-based methods change how the low-bit approximation is chosen.

3.4.1 Optimizing rounding and clipping. Adaptive rounding methods start from a simple observation: round-to-nearest minimizes the entry-wise distance between W and $\tilde{W}$, but it does not necessarily minimize the layer or block output error. A quantization grid is the discrete set of values allowed by the chosen scale and zero-point. A weight can be rounded to one of these values, and the question is whether the nearest value is always the best one for preserving the model output.

AdaRound [60] is the classical starting point, but the method predates LLM-scale PTQ. It replaces deterministic round-to-nearest with learnable rounding choices. For each weight, the method learns whether the value should be rounded up or down by minimizing a local reconstruction loss on a small set of unlabeled calibration samples.

FlexRound [39] extends this idea to LLMs by changing the rounding parameterization. Instead of modifying weights through additive rounding offsets, it uses element-wise division. This allows the method to learn a common grid scale, along with a different scaling factor for each weight. In other words, FlexRound does not only decide whether a weight rounds up or down; it also adjusts how the weight is positioned relative to the grid. The reconstruction is performed block by block. For W8A8, FlexRound reports a perplexity of 6.82 on WikiText-2 with the Llama-33B model, compared to 10.39 for AdaRound [60].

SignRound [8] makes rounding optimization cheaper by using signed gradient descent. Instead of using full gradient magnitudes, it updates rounding and clipping variables using only the sign of the gradient. The method optimizes both weight rounding and clipping thresholds in 200 steps and adds no inference-time parameters. Its gains are strongest in the 2-bit regime, where the paper reports absolute average zero-shot accuracy improvements from 6.91% to 33.22% across 11 tasks. SignRoundV2 [7] further adds adaptive mixed-precision allocation and stabilization steps such as loss filtering and scale search, targeting cases where assigning the same bit-width to every layer is too rigid.

TesseraQ [43] applies block reconstruction to ultra-low-bit LLM PTQ. Its progressive adaptive rounding uses a continuous variable between 0 and 1 to represent the choice between the lower and upper quantization grid values. During calibration, some of these variables are fixed to binary values, while the remaining continuous variables are optimized to compensate for the newly fixed rounding decisions. TesseraQ also updates dequantization scale parameters, which determine how integer codes are mapped back to the compute format. On LLaMA-2-7B with 2-bit weight-only quantization, TesseraQ reports that it improves WikiText-2 perplexity from 14.65 to 6.82 and average downstream accuracy from 50.52 to 59.27 compared with AWQ [47].

MPPQ [87] treats calibration as a joint choice of rounding, clipping, and reconstruction loss. It matches full-precision and quantized outputs at both layer and block levels, using magnitude and directional agreement. It also learns low-rank scaling factors before quantization and uses a short grid search to initialize clipping multipliers. For LLaMA-2-7B in W4A4, MPPQ reports a WikiText-2 perplexity of 8.85, compared with 12.75 for LRQuant [106] in the same setting.

3.4.2 Block-wise differentiable calibration. BRECC [42] is a foundational method in this domain, but originally developed for CNNs. The method reconstructs basic blocks rather than isolated layers, motivated by the fact that errors inside a block interact through residual paths and nonlinear operations. Its analysis also motivates later methods that optimize quantization parameters block-by-block rather than independently for each tensor.

OmniQuant [72] is a central LLM-specific method in this family. It freezes the full-precision weights and learns only a small number of quantization parameters. Its two main components are learnable weight clipping and learnable equivalent transformation. Learnable weight clipping adjusts the upper and lower clipping thresholds used before weight quantization, which reduces the effect of extreme weight values. Learnable equivalent transformation learns channel-wise transformations that preserve the full-precision computation before quantization while shifting part of the difficulty from activation quantization to weight quantization. After calibration, these learned clipping thresholds and transformation factors are rewritten into the quantized weights and scales, so OmniQuant introduces no extra inference-time modules. The method supports W4A16, W3A16, W2A16, W6A6, and W4A4 settings, and reports that LLaMA-2 models from 7B to 70B can be processed on a single A100-40G GPU within 1–16 hours using 128 calibration samples.

3.4.3 Learnable smoothing and equivalent transformations. Equivalent-transformation methods change the tensors seen by the quantizer while preserving the original full-precision matrix multiplication. SmoothQuant [89] is the key method in this research area. For a linear layer with

activation matrix $\mathbf{X}$ and weight matrix $\mathbf{W}$, it introduces a diagonal channel scale s and rewrites the multiplication as $\mathbf{XW} = (\mathbf{X} \text{diag}(\mathbf{S})^{-1}) (\text{diag}(\mathbf{S}) \mathbf{W})$. SmoothQuant reduces activation outliers through an equivalent scaling transformation. For each activation channel, it divides the activation by a scale factor and multiplies the corresponding weight channel by the same factor, preserving the full-precision product $\mathbf{XW}$ before quantization. As a simple example, $8 \times 2 = 16$ can be rewritten as $(8/2) \times (2 \times 2) = 4 \times 4 = 16$: the activation outlier is reduced, while the weight absorbs the compensating scale. This prevents the activation outlier from setting an excessively large quantization range, which would otherwise increase the scale factor and reduce the effective resolution for the remaining activation values. SmoothQuant chooses $s_j = \frac{\max(|X_j|)^\alpha}{\max(|W_j|)^{1-\alpha}}$ using calibration statistics, where α controls how much quantization difficulty is moved from activations to weights. This enables W8A8 inference with all major matrix multiplications quantized.

LRQuant [106] turns this smoothing strategy into a learned calibration problem. It uses the same idea of mathematically equivalent smoothing, but it does not keep the smoothing parameters fixed after a hand-designed initialization. It initializes the scales through logarithmic activation equalization, which is a non-linear activation-based rule designed to make activation ranges more moderate before learning begins. Then it updates the smoothing and quantization parameters with a loss that combines MSE with negative logarithm of cosine similarity. The cosine term rewards agreement in output direction, not only agreement in magnitude. LRQuant also introduces test-time adaptation: when the quantized model is evaluated on a new dataset, it updates only the learnable parameters of the last block for a few epochs, which is faster and less disruptive than recalibrating the whole model. The paper reports that, in W4A4 settings, LRQuant improves over SmoothQuant, LAE, and OmniQuant on perplexity and zero-shot tasks.

SINQ [59] addresses matrix imbalance without requiring calibration data. It introduces separate row and column scale vectors and uses a dampened Sinkhorn-Knopp-style procedure in log space to normalize row and column standard deviations before quantization. This balancing reduces the influence of weight outliers while avoiding the increase in row-wise kurtosis that can result from direct column scaling. The resulting scales can be applied with little runtime overhead, and the additional column scales can in some settings be absorbed into adjacent operations.

Optimization-based transformation methods are most useful when the low bit-width is not the only problem. Activation outliers, poor clipping ranges, and bad initial scales can make even 4-bit or 8-bit quantization fail. These methods improve PTQ by changing the tensor distribution before rounding or by learning the rounding and clipping parameters directly from calibration examples.

Key takeaway. Optimization-based PTQ treats quantization as calibration-time learning. Its central idea is to choose rounding decisions, clipping ranges, scales, smoothing factors, and equivalent transformations by matching full-precision and quantized outputs on a small calibration set. The calibration samples often do not need labels, because the full-precision model supplies the target outputs. The main advantage is stronger accuracy at low bit-widths, especially when round-to-nearest, fixed clipping, or hand-designed scaling fail. The main cost is calibration time and sensitivity to the chosen reconstruction objective, calibration data, and whether the learned changes can be rewritten into ordinary low-bit weights and quantization scales for inference.

Table 4 shows that modern PTQ is best understood as calibration-guided error control. Weight quantization is the common approach, but activation and KV-cache quantization appear only in a smaller set of methods, due to their greater runtime sensitivity. A second pattern is the separation between accuracy mechanisms and deployment cost. Codebooks, sparse preserved weights, online transforms, and extra high-precision terms can improve low-bit fidelity, but they

Table 4. Classification of representative PTQ methods for LLMs. ✓ if the aspect is required by the complete pipeline used in the paper including default backends and optimization stages (not only the novel contribution), and ✗ indicates that the aspect is not applied.

Method	Target tensor			Calibration				Intervention				Runtime				Evidence					
	Weights	Activations	KV cache	Range stats	Activation stats	Reconstruction	Hessian / curvature	Learned params	Compensation	Transform / rotation	Salience	Codebook	Rounding / clipping	Offline / fused	Online transform	Mixed precision	Lookup / sparse	Extra FP term	Sub-4 bit	W+A setting	Hardware eval.
Compensation-based																					
GPTQ [20]	✓	✗	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
SEPTQ [48]	✓	✗	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
LQER [101]	✓	✓	✗	✗	✓	✓	✗	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
QERA [102]	✓	✗	✗	✗	✓	✓	✗	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✗
AQLM [18]	✓	✗	✗	✗	✓	✓	✓	✓	✗	✗	✗	✓	✗	✓	✗	✗	✗	✗	✓	✗	✓
VPTQ [51]	✓	✗	✗	✗	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓	✗	✓
Rotation-based																					
QuIP [6]	✓	✗	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
QuIP# [81]	✓	✗	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
QuaRot [5]	✓	✓	✓	✗	✓	✓	✓	✗	✓	✓	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
SpinQuant [53]	✓	✓	✓	✗	✓	✓	✓	✗	✓	✓	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
FlatQuant [78]	✓	✓	✓	✗	✓	✓	✗	✗	✗	✓	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
RotateKV [76]	✗	✗	✓	✗	✓	✗	✗	✗	✗	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✗	✓
Salience-based																					
LLM.int80 [13]	✓	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
AWQ [47]	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✓	✓	✓	✗	✗	✗	✗	✓	✗	✓
OWQ [36]	✓	✗	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
SpQR [15]	✓	✗	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	✗	✓
Slim-LLM [33]	✓	✗	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
PB-LLM [100]	✓	✗	✗	✗	✓	✓	✓	✓	✓	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	✗	✗
Optimization-based																					
FlexRound [39]	✓	✓	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✓	✓
SignRound [7]	✓	✗	✗	✗	✓	✓	✓	✓	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✓
TesseraQ [43]	✓	✓	✗	✓	✓	✓	✓	✓	✗	✗	✓	✓	✓	✓	✗	✗	✗	✗	✓	✓	✓
OmniQuant [72]	✓	✓	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✓	✓
SmoothQuant [89]	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗	✓	✓
LRQuant [106]	✓	✓	✗	✗	✓	✓	✓	✓	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✓	✓
MPPQ [87]	✓	✓	✗	✗	✓	✓	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✓	✓	✓

also complicate inference. Methods whose calibration choices are rewritten into stored low-bit weights and quantization scales are generally easier to deploy.

4 Sensitivity of Transformer Components to Quantization

Component sensitivity refers to the degradation caused by quantizing a specific tensor or module while the rest of the computation is kept fixed or calibrated [47]. The component perspective is useful because PTQ error is governed by where the error is introduced and how it is amplified by the transformer computation [20, 89]. See Fig. 7.

A useful first distinction is between weights, activations, and cached states. Weight tensors are fixed after training and can be quantized offline. Their distributions are often easier to fit with per-channel or group-wise quantization. Activations are generated at runtime and depend on the prompt, token position, and layer. They often contain outliers concentrated in a small set of channels. These outliers increase the quantization range and reduce the effective resolution assigned to the remaining activation values. Cached key-value states add another difficulty: they grow with context length and are reused during decoding, so their quantization error can affect many subsequent tokens. Large weight matrices determine most of the compression benefit, but they are not always the most fragile tensors. In many LLMs, low-bit quantization is particularly sensitive to activation outliers, attention-score perturbations, KV-cache errors, normalization outputs, and errors in the final logits. Component-aware PTQ methods exploit this non-uniform sensitivity by spending

Table 5. Summary of PTQ Strategy Categories

Category	Core Mechanism	Calibration Requirements	Runtime Implications	Primary Trade-off
Compensation	Mitigates quantization error through residual correction, low-rank updates, sparse outlier preservation, or second-order error propagation.	Typically requires calibration activations; many methods estimate reconstruction error or curvature information.	Usually preserves the original model structure with minimal runtime overhead, although extra correction terms may increase memory or computation.	Improved low-bit accuracy at the cost of additional storage, calibration complexity, or auxiliary parameters.
Rotation	Applies orthogonal transformations to redistribute activation and weight magnitude and reduce outliers before quantization.	Requires calibration data when rotations are optimized, while fixed rotations may require little or no optimization.	May introduce online transform operations unless fused into model weights.	Better activation and KV-cache quantization, but potential runtime overhead from transformation operators.
Salience	Identifies important weights, channels, or tokens and allocates higher precision or special treatment to them.	Relies on activation statistics, Hessian information, or outlier analysis to estimate importance.	Often introduces mixed-precision execution or sparse high-precision components.	Preserves critical information efficiently but may reduce hardware regularity.
Optimization	Directly optimizes quantization parameters such as scales, clipping thresholds, rotations, or rounding decisions using calibration objectives.	Typically requires calibration data and iterative optimization.	Usually incurs no runtime overhead once optimized parameters are fused.	Higher calibration cost in exchange for improved quantization accuracy.

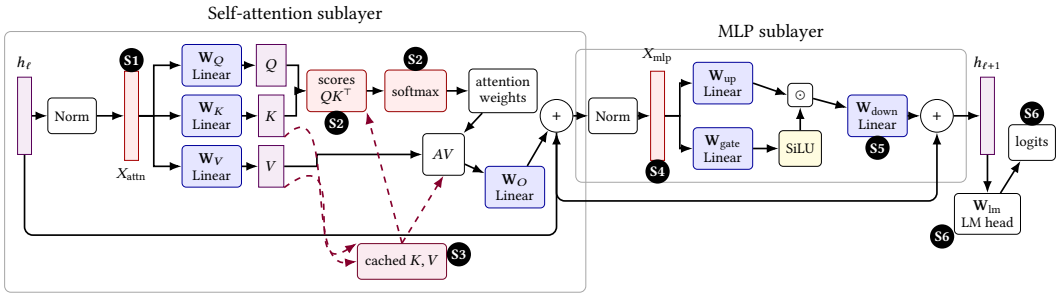


Fig. 7. Sensitivity map of transformer components under PTQ. Weight matrices in the attention and MLP sublayers are often amenable to low-bit quantization with per-channel or group-wise calibration. On the other hand, activations produced by RMSNorm/LayerNorm, attention-score computation, cached key-value states, MLP outputs returned to the residual stream, and the final projection to vocabulary logits are more fragile. **S1**: activation outliers after RMSNorm/LayerNorm can inflate the quantization scale. **S2**: quantization errors in Q or K can perturb QK^T , and softmax can amplify these perturbations into attention-routing changes. **S3**: quantization error in the KV cache can be reused during later decoding steps. **S4**: MLP input activations can contain outliers that reduce quantization resolution. **S5**: MLP output error is returned to the residual stream and passed to later blocks. **S6**: logit errors can change token probabilities.

calibration effort, correction terms, or higher precision only where quantization error changes model behavior most.

4.1 Attention Projections

The self-attention sublayer is one of the most sensitive parts of a decoder-only transformer. Given an input activation matrix X , the sublayer forms $Q = XW_Q$, $K = XW_K$, and $V = XW_V$, computes attention scores from QK^T , aggregates values through V , and projects the result back to the residual stream through W_O . Quantization errors in W_Q and W_K perturb the attention scores directly, and the softmax operation can amplify these perturbations. Errors in W_V affect the token information retrieved by attention, while errors in W_O propagate the attention output into the hidden state used by later blocks. The important point is that attention quantization can change how information is “routed”, as follows. In an MLP layer, quantization error mainly changes the numerical representation of features. In self-attention, quantization error in Q or K can change which previous tokens receive attention. This makes attention-score computation more fragile than ordinary linear reconstruction error suggests. The most fragile part is often the input activation to

the query, key, and value projections. These activations are produced by RMSNorm/LayerNorm and can contain large channel outliers. If such activations are quantized to a single coarse range, a few extreme values determine the scale factor, reducing precision for most entries. This is why activation-aware methods, smoothing transformations, and salience-based channel protection are frequently applied around the attention projections. Low-bit quantization of attention weights is often feasible with appropriate calibration, but low-bit quantization of the activations entering $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ is substantially more difficult.

4.2 MLP Layers

The MLP layers contain a large fraction of transformer parameters (*i.e.*, 70% for Llama 3.1-8B) and are therefore important for compression. MLP weight matrices are often good targets for aggressive weight-only quantization because they are large, fixed, and amenable to per-channel or group-wise scaling. In LLaMA-style decoder blocks, the MLP sublayer uses a gated feed-forward form. Given an input activation matrix $\mathbf{X}$, it computes an expanded representation $\mathbf{X}\mathbf{W}_{\text{up}}$, a gating representation $\text{SiLU}(\mathbf{X}\mathbf{W}_{\text{gate}})$, multiplies them elementwise, and then maps the result back to the residual dimension through $\mathbf{W}_{\text{down}}$. Thus, $\mathbf{W}_{\text{up}}$ and $\mathbf{W}_{\text{gate}}$ are expansion projections, while $\mathbf{W}_{\text{down}}$ is the contraction projection back to the model hidden size. The MLP expansion projections $\mathbf{W}_{\text{up}}$ and $\mathbf{W}_{\text{gate}}$ receive the layer normalization output. Large outliers in this input can dominate the quantization range and reduce precision for the remaining activation values. The down projection aggregates the transformed hidden features back into the residual stream. As a result, MLP weight quantization often works well at 4 bits, but lower-bit weight-and-activation settings usually need clipping, smoothing, learned rounding, or block-wise reconstruction.

4.3 KV Cache

The KV cache has a different sensitivity profile from static weights. During decoding, each layer stores keys and values from previous tokens. These cached states are reused at every later decoding step, so cache quantization affects both memory use and generation behavior. Unlike weight quantization, KV-cache quantization acts on runtime tensors whose errors persist across future token generation. The **key cache** is especially sensitive because keys participate in attention-score computation. A small distortion in keys can change which past tokens receive attention. The **value cache** affects the content retrieved after attention weights have been computed. Thus, keys affect where the model looks, while values affect what information is retrieved [52].

Due to this asymmetry, keys and values are often quantized differently. Keys are better handled with channel-aware quantization when their outliers are persistent across the same key dimensions, while values are often better handled with token-aware quantization because their large entries vary more by token.

4.4 Normalization Layers and Residual Paths

Normalization layers and residual paths are small in parameter count but large in effect. Layer normalization shapes the activations consumed by attention and MLP projections. Quantizing these operations aggressively saves little memory, but errors near them can affect the inputs of large matrix multiplications. In practice, PTQ methods often keep normalization parameters in higher precision or absorb their effects into neighboring linear layers through equivalent transformations [59, 89]. A quantized projection affects both its own output and the hidden state that later blocks receive. Therefore, an error introduced in one block can be carried forward and modified by many later blocks. Block-wise calibration and rotation-based methods have been partly developed to address this issue. They either preserve the behavior of a transformer block or redistribute outlier energy before quantization, thereby reducing distortion in the residual stream.

4.5 Output Path and Mixed-Precision Implications

The embedding matrix and LM head are often quantized in higher bits [20]. They interact directly with the vocabulary space, and small distortions in the output projection can change token probabilities. Although these tensors may not dominate the total parameter count in large models, the final logits determine generation quality. This makes the output path a high-sensitivity region, especially for instruction following, reasoning, and long-form generation. The broader implication is that uniform bit-width assignment is typically not used. Mixed-precision PTQ is therefore the natural response to non-uniform sensitivity. This also motivates the role of calibration in the next section as a sensitivity probe; it reveals which activations contain outliers, which channels are salient, which blocks reconstruct poorly, and which tensors need protection under low-bit quantization.

Key takeaway. Large attention and MLP weight matrices dominate model storage and are often good low-bit quantization targets. The most fragile errors often arise in runtime quantities: normalization outputs with channel outliers, attention scores formed from Q and K , KV-cache states reused during decoding, MLP outputs returned to the residual stream, and logits.

5 Calibration Strategies

In PTQ, calibration is the post-training step that estimates quantizer parameters for a fixed pre-trained model [47, 72]. A calibration set is a small collection of unlabeled inputs; these examples are passed through the model to record activations and full-precision layer or block outputs. The set should reveal which activations contain persistent channel outliers [89], which weight channels are important under the observed activations, which blocks reconstruct poorly, and which transformations reduce quantization difficulty [72, 106]. Thus, weak calibration estimates only numerical ranges, while stronger calibration acts as a sensitivity probe for learned rounding choices [42, 60], clipping thresholds and transformations [72], and optimal bit allocation [8, 43]. Table 6 summarizes PTQ calibration strategies.

5.1 Range-Based Calibration

The simplest calibration rule selects the quantization range based on observed tensor extrema. In min-max calibration, the scale is determined by the smallest and largest observed values, optionally per tensor, per channel, or per group [25]. Percentile clipping modifies the same idea by discarding a small fraction of extreme values before fitting the grid [13]. These procedures are fast, hardware-friendly, and still useful as initialization, but they are fragile in LLMs because activation outliers can be systematic rather than rare [89]. At low bit-width, a single large channel can expand the range and waste most of the available levels on infrequent values.

5.2 Distributional Calibration

Distributional calibration uses more information than the minimum and maximum of a tensor. Histogram-based methods estimate the empirical distribution of tensor values and choose quantization parameters that preserve this distribution after discretization. KL-divergence calibration follows this principle by selecting a quantization range to reduce the mismatch between the original and quantized histograms [31]. This is a stronger criterion than min-max calibration because it uses the shape of the distribution. The main limitation of this approach is that it treats a tensor mostly as a collection of values. LLM quantization, on the other hand, often needs a functional view: an error matters because of where it enters the transformer computation. This motivates reconstruction-based calibration, where the objective is not only to approximate the tensor but to preserve the output produced by that tensor inside the network.

5.3 Reconstruction-Based Calibration

Reconstruction-based calibration sets quantization parameters by matching the output of a chosen part of the network on calibration inputs. We call this chosen part the *calibration scope*. The scope may be a single linear projection, an attention or MLP sub-block, or a full transformer block. The scope fixes the computation whose error is minimized: a weight-level objective measures $|\mathbf{W} - \tilde{\mathbf{W}}|_F^2$, while a reconstruction objective measures $|\mathbf{X}\mathbf{W} - \mathbf{X}\tilde{\mathbf{W}}|_F^2$.

Layer-wise calibration. Layer-wise reconstruction chooses quantizer parameters ϕ so that the dequantized low-bit matrix $\tilde{\mathbf{W}}_\phi$ gives a close output $\mathbf{X}\tilde{\mathbf{W}}_\phi$. AdaRound [60] uses this view to select rounding decisions from calibration data, and GPTQ [20] scales the same principle to LLM weights with a Hessian-aware layer-wise procedure.

Block-wise calibration. Block-wise reconstruction uses a full transformer block as the calibration scope. Let $\mathcal{F}^{(\ell)}$ denote block ℓ with parameters θ_ℓ . Calibration chooses quantizer parameters $\tilde{\Theta}_{\ell, \phi_\ell}$ by minimizing $\|\mathcal{F}^{(\ell)}(\mathbf{X}^{(\ell)}; \Theta_\ell) - \mathcal{F}^{(\ell)}(\mathbf{X}^{(\ell)}; \tilde{\Theta}_{\ell, \phi_\ell})\|_F^2$ on the same calibration inputs. This scope includes the composed effect of attention, MLP projections, nonlinear gating, normalization, and residual addition. BRECQ [42] introduced block reconstruction as a PTQ calibration scope, and LLM methods such as OmniQuant [72], SignRound [7], and TesseraQ [43] use block outputs as the signal for estimating their quantization parameters.

Scope selection. The calibration scope sets the cost–fidelity trade-off. A single linear layer is cheap to calibrate and works well when the dominant error is local to one weight matrix. A transformer block is more expensive because it requires the full block computation during calibration, yet it tracks errors after they pass through attention, gating, normalization, and residual addition. Reconstruction-based calibration is, therefore, a choice about where the quantizer is asked to preserve behavior.

5.4 Activation-Aware Calibration

Activation-aware calibration uses the activations observed on a calibration set to decide how quantization parameters should be assigned. In LLMs, this step is essential because activations are runtime tensors: their ranges depend on the prompt, token position, and linear layer, and they often contain channel outliers that are much harder to quantize than weights [13, 89]. Thus, activation-aware calibration asks which activation channels make weight or output error more damaging. SmoothQuant [89] is the canonical example. For a linear layer with activation matrix $\mathbf{X}$ and weight matrix $\mathbf{W}$, it uses calibration activations to choose a diagonal scale matrix $\mathbf{S}$ and rewrites the computation as $\mathbf{X}\mathbf{W} = (\mathbf{X}\mathbf{S}^{-1})(\mathbf{S}\mathbf{W})$. The product is unchanged before quantization, but the activation tensor given to the quantizer has a smaller range. The calibration problem is therefore to choose $\mathbf{S}$: too little scaling leaves activation outliers intact, while too much scaling moves excessive range into the weights. OmniQuant [72] keeps the same equivalent-transformation principle but estimates the transformation through block-wise calibration, together with learned clipping thresholds.

Activation statistics can also identify which weight channels deserve protection. AWQ [47] uses calibration activations to score weight channels: a weight perturbation is more harmful when it is multiplied by large input activations. In this view, activations serve as a salience probe. LRQuant [106] follows the same activation-centered logic by learning smoothing parameters and using an output-similarity objective to reduce calibration overfitting.

5.5 Calibration Data Selection

GPTQ, AWQ, OmniQuant, SignRound, and TesseraQ show that a few hundred unlabeled sequences are often sufficient for many weight-only or block-wise PTQ settings [8, 20, 43, 47, 72]. For general

Table 6. Summary of Calibration Strategies for PTQ

Category	Core Idea	Strengths	Key Limitations
Min–Max	Determines quantization parameters directly from the observed minimum and maximum values of a tensor.	Requires no optimization or threshold search and can be applied with minimal calibration overhead.	Highly sensitive to outliers, which may compress the effective dynamic range available to most values.
Percentile-Based	Sets quantization ranges using percentile thresholds while discarding a small fraction of extreme values.	Reduces the influence of outliers and improves utilization of available quantization levels.	Requires selection of clipping percentiles, whose optimal values may vary across models and tensors.
Histogram / KL	Builds value histograms and selects clipping thresholds by minimizing distribution divergence or entropy-based criteria.	Preserves tensor distributions more effectively than extrema-based methods and can improve low-bit quantization.	Introduces additional calibration cost due to histogram construction and threshold optimization.
MSE-Optimal	Searches quantization parameters that minimize reconstruction error between full-precision and quantized tensors.	Directly optimizes numerical fidelity and often produces well-calibrated quantization ranges.	Minimizing reconstruction error does not necessarily maximize downstream task accuracy.
Activation-Aware	Uses activation statistics to guide weight scaling, clipping, or quantization parameter allocation.	Allocates quantization capacity according to the importance of activation channels and features.	Requires representative calibration data and additional statistics collection.
Granularity-Aware	Applies quantization at different granularities, such as per-tensor, per-channel, group-wise, or block-wise scaling.	Adapts quantization parameters to local tensor characteristics and reduces quantization error.	Increases metadata storage requirements and may complicate hardware implementation.
Dataset Selection	Constructs calibration sets using representative, diverse, or perplexity-based sample selection strategies.	Produces activation distributions that better match deployment conditions and improves calibration quality.	Depends on data availability and introduces sample-selection overhead.
Zero-Shot	Derives quantization parameters solely from model weights or analytical statistics without calibration data.	Eliminates dependence on calibration datasets and avoids data-access or privacy concerns.	Cannot adapt quantization parameters to real activation distributions and may be less reliable at very low bit widths.

language modeling, random text segments can be adequate for estimating weight and activation statistics [72]. For instruction-tuned models, long-context deployment [106], code, mathematics, or reasoning-heavy workloads, calibration prompts should match the structure and length of the target use case because activation and attention patterns change across domains.

Once the relevant activation ranges, outlier channels, and sensitive blocks are observed, additional calibration samples may add limited benefit [47, 72]. The more serious risk is mismatch: a quantizer calibrated on short generic text can fail on long prompts, dialogue, code, or mathematical reasoning because the runtime activation distribution has changed [87, 106]. LRQuant explicitly treats this as a generalization problem by adding a robustness-oriented loss and test-time adaptation for unseen data [106].

Optimization-based calibration can also overfit from a small calibration set and lose robustness on unseen examples [87, 106]. Calibration and validation are therefore separated: calibration data estimates quantization parameters, while held-out tasks test whether the quantized model preserves behavior [20].

5.6 Zero-Shot and Data-Free Calibration

Zero-shot and data-free calibration specify the information source used to estimate quantization parameters. Zero-shot calibration refers to calibration from unlabeled real text. Data-free calibration estimates the same parameters from model-generated samples or parameter-only statistics. This self-calibration generates synthetic sequences from the model and then applies ordinary statistics-based or reconstruction-based calibration. Parameter-only or calibration-free methods estimate quantization decisions from the pretrained weights alone. EasyQuant [79] isolates weight outliers and optimizes the quantization range of the remaining weights using weight reconstruction error,

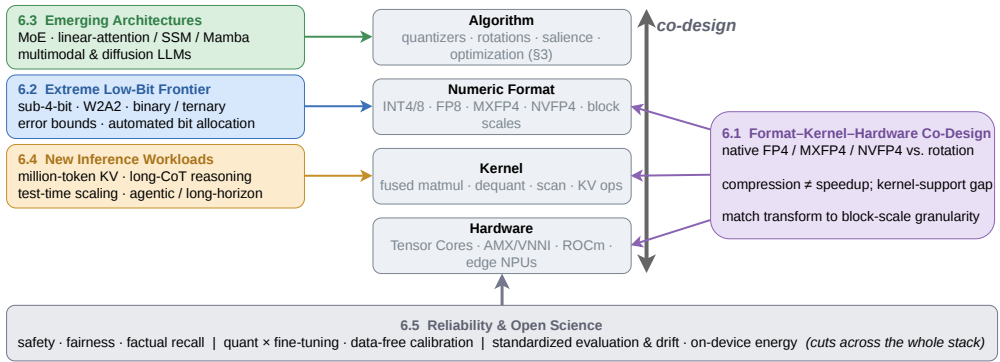


Fig. 8. Overview of the research frontier. We read PTQ as a stack of four layers—algorithm, numeric format, kernel, and hardware—that must be optimized jointly (the vertical “co-design” arrow). Five clusters of open problems sit around the stack: emerging architectures (§6.3), the extreme low-bit frontier (§6.2), and new inference workloads (§6.4) bear mainly on the algorithm and the formats it targets; format–kernel–hardware co-design (§6.1) couples the lower three layers; and reliability and open science (§6.5), shown as the band beneath the stack, cut across all of them. Cluster colors and numbering match §6.1–§6.5.

without calibration inputs. AdpQ [24] uses Adaptive LASSO-style soft-thresholding to identify salient weights and then quantizes outlier and non-outlier weights separately. These methods replace activation-based calibration with assumptions about weight distributions, outlier structure, and the relation between weight reconstruction and model fidelity.

Key takeaway. Calibration becomes more important as PTQ moves from fixed weights to runtime tensors. Weight-only methods can often estimate quantizers from parameter statistics. Activation and cache quantization depend on prompt-specific ranges, channel outliers, and context length. Ideally, representative inputs should reflect these runtime conditions, and their quality becomes especially important for low-bit activation quantization.

6 Research Challenges and Future Directions

PTQ for LLMs sits on a stack of four interacting layers: a quantization algorithm, a numeric format, a kernel, and the hardware beneath them. The hardest open problems appear when one linear layer changes and the others must adapt, so we organize the frontier by *which part of the stack is changing*. Figure 8 maps the five clusters onto the stack. Where relevant, we relate each problem to the error-control families of Section 3.

6.1 The Format–Kernel–Hardware Co-Design Frontier

Accelerators now ship native low-bit floating-point formats with block-level scales, so the central question has become which algorithm fits the block layout the hardware defines. This layer increasingly decides what can be deployed at all.

Native FP4 microscaling formats and the rotation–format interaction. Blackwell-class hardware natively supports MXFP4 (E2M1 data, 32-element blocks, power-of-two block scales) and NVFP4 (16-element blocks with FP8 block scales) [17, 64]. Both clash with the rotation-based PTQ: NVFP4’s small group size provably neutralizes traditional outlier-mitigation transforms, and MXFP4’s power-of-two scale quantization degrades accuracy on its own [17]. Global Hadamard rotation also fights the per-block exponent, since one outlier inflates the shared scale of its block; recovering accuracy requires block-wise rotation aligned to the MX group size [74]. Co-tuning rotation and scaling granularity with the block layout, and allocating the scarce FP8 block-scale bits,

remain open questions. Matching the transform to the format pays off in practice: format-aware GPTQ (MR-GPTQ) reports up to $2.2\times$ end-to-end speedup on B200 and $4\times$ on RTX5090 [17].

Compression does not imply speedup: the kernel-support gap. High compression ratios rarely translate into proportional latency gains: irregular group sizes, mixed precision, and dequantization fragment memory access, and decode is bound by memory bandwidth rather than compute. Many published methods also lack a fused kernel on mainstream hardware (NF4 and variable-group INT4 run on few devices, and support diverges across NVIDIA, AMD, and Intel), so frameworks such as Marlin [21], TensorRT-LLM [63], ExLlama [82], and BitsAndBytes [13] realize only a subset of the proposed formats. New methods should therefore target a realizable fused kernel and report on-target latency and energy alongside bit-width (Section 6.5).

6.2 Pushing the Accuracy Frontier at Extreme Low Bit-Width

Below roughly four bits, rounding error stops being a small perturbation: it rivals the signal in sensitive directions, the assumptions behind first-order and Hessian-based compensation weaken, and outlier handling comes to dominate the loss.

Extreme low-bit quantization. Recent PTQ methods have extended quantization to ternary weights without retraining through dedicated ternarization methods [95] and extreme low-bit vector quantization [51]. Most results in this regime concern weight-only quantization. Joint W2A2 quantization remains an open problem for general-purpose LLMs. Performance degradation at low precision also varies with model size, large models tolerate 4-bit weights while small models fall off a cliff [38]. Further work is needed on codebook-based reconstruction at extreme bit-widths, incoherence-based transformations for 2-bit quantization, salience-based mixed 2/4-bit allocation, and distillation-based recovery.

Theoretical error bounds and guarantees. The PTQ field runs almost entirely on empirical evidence. There are no tight bounds linking a method’s quantization error to end-task degradation, and no provable guarantees for rotation-based incoherence. The difficulty is that linear layer-wise reconstruction loss is a loose proxy for downstream loss in a non-linear, emergent system. Error-propagation analyses that model how per-layer error compounds through transformer blocks are an early step [4]; predictive end-to-end bounds do not yet exist.

Automated bit allocation and joint compression. Bit-width, numeric format, and group size are still commonly selected through manual tuning, although these choices interact with model sensitivity and hardware constraints. Automated allocation methods could instead select these configurations through differentiable optimization or search, using salience estimates and the kernel constraints discussed in Section 6.1. A related problem concerns the joint design of quantization with other compression methods. Applying quantization, pruning, and low-rank factorization sequentially can accumulate approximation error. Recent methods therefore optimize these forms of compression jointly, combining quantization with sparsity [28] or with both sparsity and low-rank approximation [58].

6.3 Quantizing Emerging Architectures

The component-sensitivity findings of Section 4 were established largely on softmax-attention transformers. The architectures now reaching the frontier (e.g., sparse experts [61], recurrent state-space models [66], and multimodal and diffusion decoders [98]) relocate the sensitive components, so their quantization profiles must be studied from scratch.

Mixture-of-Experts. MoE models add expert sparsity, heterogeneous per-expert activation statistics, and a precision-fragile router [19]: a shared scale fits no expert well, rarely activated experts receive poor calibration coverage, and router noise flips token assignments. Expert-aware optimization that aligns routing consistency, balances calibration across experts, and smooths activation

outliers recovers 1.15–13.81% average accuracy under aggressive W2–W4 settings [22]. Because expert usage is heavy-tailed and shifts across workloads, DynaExq [10] tracks router statistics during inference and keeps frequently used experts at higher precision.

Linear-attention, state-space, and hybrid models. Sub-quadratic architectures [96] replace softmax attention with recurrent state updates, so error in the recurrent state compounds along the sequence, and a fixed-size reused state takes the place of the KV cache. State-space models (SSMs) exhibit activation-outlier channels much like transformers, so naive quantization fails [66] and dedicated PTQ recipes are needed [9]. Two questions remain: quantizing the recurrent update with explicit error bounds, especially for gating and decay parameters, and allocating precision in attention–SSM hybrids whose layer types differ in sensitivity.

Multimodal and diffusion LLMs. Vision, audio, and video language models mix modalities with different dynamic ranges and rely on quantization-sensitive cross-modal attention, so a single quantizer fit on pooled data may let the insensitive modality dominate the scales. Modality-balanced quantization, which allocates by per-modality sensitivity, recovers up to 4.4% at W3A16 and 11.6% at W4A8 over prior VLM PTQ [41]; it builds on cross-layer dependencies that layer-wise calibration ignores [84] and extends to MoE-VLMs [68]. In diffusion LLMs [62], generation proceeds through repeated denoising under a masking schedule, so calibration must use activations from the denoising timesteps used at inference [46, 103].

6.4 Quantization under New Inference Workloads

The same quantized model may behave differently as the workload lengthens. Million-token contexts, long chains of thought, and multi-step agentic trajectories each turn a negligible per-step error into a long-horizon failure that short-task metrics may miss.

Million-token KV cache. As contexts grow past 128K toward 1M tokens, the KV cache becomes the binding constraint on serving cost, and early tokens re-attended thousands of times have their quantization error amplified by reuse. Preserving attention sinks beyond the first token is decisive: on LLaMA2-70B with 4-bit KV, keeping a handful of sink tokens reduces perplexity by 163.2 relative to keeping only the leading tokens [77]. Rotation absorbs key-channel outliers and enables accurate 2-bit KV [76], and chunk-adaptive mixed precision targets long contexts directly [80]. The field still lacks a KV quantizer with error guarantees over sequence length.

Reasoning, long chain-of-thought, and test-time scaling. Models whose value lies in long generation are exactly where per-step error accumulates. W8A8 and W4A16 are near-lossless, but below that, reasoning degrades sharply and unevenly across task difficulty [49]. Quantization also changes generation length: aggressive 3-bit weight-only quantization inflates chain-of-thought from 19K to 38.8K tokens while accuracy falls from 17.5% to 4.5% ($\rho = -0.73$ between accuracy loss and length increase) [54]. Two directions remain open: bounding rounding error along the generation trajectory, and evaluating quantized models on long-form reasoning benchmarks.

Agentic and long-horizon workloads. Agentic deployments call the model repeatedly, use tool outputs, and carry context across steps. Small PTQ errors can therefore affect later actions, tool arguments, or stored assumptions, even when static metrics remain acceptable [16]. PTQ evaluation for agents should move toward multi-step tasks that measure tool-call validity, constraint retention, and recovery from earlier errors.

6.5 Reliability, Process, and Open Science

The final cluster concerns trust in the stack as a whole. Quantization can silently shift behaviors that accuracy and perplexity never probe, and the field’s evaluation practices make such shifts hard to detect. These concerns cut across every other cluster, which is why Figure 8 draws them as a band wrapping the stack.

Safety, fairness, and factual recall. Quantization shifts activation distributions and can change behavior or fairness while leaving perplexity and accuracy untouched [67, 69, 92]. Safety-preserving PTQ via a contrastive alignment loss [86] and critical-weight protection [3] show that the weights carrying these behaviors form a distinct, protectable subset; quantization also degrades factual recall and makes model explanations less trustworthy and coherent [85]. Progress requires making these evaluations routine alongside accuracy and identifying which components carry the behaviors so the salience family can protect them.

Quantization $\times$ fine-tuning and data-free calibration. PTQ and fine-tuning are usually treated as separate stages: quantizing before fine-tuning fixes the low-bit representation before task adaptation, while fine-tuning low-rank adapters on a quantized base [14] adapts to a quantizer chosen earlier. Each stage reshapes what the other sees, and joint methods that choose quantization and adaptation parameters together remain underdeveloped. Separately, fully data-free PTQ is attractive when calibration inputs are unavailable but still trails data-driven calibration on factual recall and long-context behavior; self-generated calibration data [88] and broader zero-shot PTQ [34] are promising but immature.

Standardized evaluation and on-device cost. PTQ results are difficult to compare across heterogeneous setups, and benchmark contamination compounds the problem [27, 38, 50, 75]. A unified protocol should cover long-context, reasoning, and safety tasks *and* report on-target latency and energy on named hardware. Most papers report accuracy and bit-width, yet for edge deployment under tight power budgets, joules per token, carbon cost, and hardware-compatible formats matter just as much.

7 Conclusion

This survey presented PTQ for LLM inference from an error-oriented perspective. We categorized PTQ methods by how they mitigate quantization error: compensation corrects errors after low-bit approximation, rotation redistributes quantization difficulty, salience allocates precision to important weights or channels, and optimization learns quantizer parameters from calibration data. This taxonomy emphasizes that PTQ performance depends not only on bit-width but also on where errors arise within the transformer and how calibration determines scales, clipping thresholds, rotations, salience scores, rounding strategies, codebooks, and bit allocations. Future PTQ research will focus on the settings where current methods remain fragile: sub-4-bit weights, low-bit activations, KV-cache quantization for long contexts, reasoning workloads, mixture-of-experts and multimodal models, and hardware-native FP4 formats. Progress there requires methods that respect both model sensitivity and hardware capabilities, and evaluations that report the quantized components, calibration protocol, numeric format, target hardware, latency, memory, and model quality, since bit-width alone cannot characterize a quantized LLM.

References

- [1] Harshavardhan Adep, Zhanpeng Zeng, Li Zhang, and Vikas Singh. 2024. FrameQuant: flexible low-bit quantization for transformers. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*.
- [2] Mohammad Sadegh Akhondzadeh, Aleksandar Bojchevski, Evangelos Eleftheriou, and Martino Dazzi. 2025. KurTail: Kurtosis-based LLM Quantization. *arXiv preprint arXiv:2503.01483* (2025).
- [3] Muhammad Alif Al Hakim, Alfian Farizki Wicaksono, and Fajri Koto. 2026. Preserving Fairness and Safety in Quantized LLMs Through Critical Weight Protection. In *Findings of the Association for Computational Linguistics: ACL 2026*. 19831–19855.
- [4] Yamato Arai and Yuma Ichikawa. 2025. Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems, NeurIPS*.
- [5] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. 2024. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. In *The Thirty-eighth*

- Annual Conference on Neural Information Processing Systems*.
- [6] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. 2023. QuIP: 2-Bit Quantization of Large Language Models With Guarantees. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
 - [7] Wenhua Cheng, Weiwei Zhang, Heng Guo, Haihao Shen, and Zaner Ma. 2025. SignRoundV2: Toward Closing the Performance Gap in Extremely Low-Bit Post-Training Quantization for LLMs. *arXiv preprint arXiv:2512.04746* (2025).
 - [8] Wenhua Cheng, Weiwei Zhang, Haihao Shen, Yiyang Cai, Xin He, Kaokao Lv, and Yi Liu. 2024. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP*.
 - [9] Hung-Yueh Chiang, Chi-Chih Chang, Natalia Frumkin, Kai-Chiang Wu, Mohamed S. Abdelfattah, and Diana Marculescu. 2025. Quamba2: a robust and scalable post-training quantization framework for selective state space models. In *Proceedings of the 42nd International Conference on Machine Learning ICML*. 17 pages.
 - [10] Kexin Chu, Dawei Xiang, Zixu Shen, Yiwei Yang, Zecheng Liu, and Wei Zhang. 2025. Dynamic Expert Quantization for Scalable Mixture-of-Experts Inference. *arXiv preprint arXiv:2511.15015* (2025).
 - [11] Patrik Czakó, Gábor Kertész, and Sándor Szénási. 2025. Addressing Activation Outliers in LLMs: A Systematic Review of Post-Training Quantization Techniques. *IEEE Access* 13 (2025), 81917–81932.
 - [12] Patrik Czakó, Gábor Kertész, and Sándor Szénási. 2025. SmoothRot: Combining channel-wise scaling and rotation for quantization-friendly llms. In *2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 6461–6466.
 - [13] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*.
 - [14] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Thirty-seventh Conference on Neural Information Processing Systems*.
 - [15] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. 2024. SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression. In *The Twelfth International Conference on Learning Representations, ICLR*.
 - [16] Peijie Dong, Zhenheng Tang, Xiang Liu, Lujun Li, Xiaowen Chu, and Bo Li. 2025. Can Compressed LLMs Truly Act? An Empirical Evaluation of Agentic Capabilities in LLM Compression. In *Proceedings of the 42nd International Conference on Machine Learning, ICML*.
 - [17] Vage Egiazarian, Roberto L. Castro, Denis Kuznedelev, Andrei Panferov, Eldar Kurtic, Shubhra Pandit, Alexandre Noll Marques, Mark Kurtz, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2026. Bridging the Gap Between Promise and Performance for Microscaling FP4 Quantization. In *The Fourteenth International Conference on Learning Representations*.
 - [18] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme Compression of Large Language Models via Additive Quantization. In *Forty-first International Conference on Machine Learning, ICML*.
 - [19] Elias Frantar and Dan Alistarh. 2023. Qmoe: Practical sub-1-bit compression of trillion-parameter models. *arXiv preprint arXiv:2310.16795* (2023).
 - [20] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
 - [21] Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefer, and Dan Alistarh. 2025. MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2025, Las Vegas, NV, USA, March 1-5, 2025*.
 - [22] Zhongqian Fu, Tianyi Zhao, Ning Ding, Xianzhi Yu, Xiaosong Li, Yehui Tang, and Yunhe Wang. 2025. EAQuant: Enhancing Post-Training Quantization for MoE Models via Expert-Aware Optimization. *arXiv preprint arXiv:2506.13329* (2025).
 - [23] Advait Gadhikar, Riccardo Grazi, and James Hensman. 2025. OptRot: Mitigating Weight Outliers via Data-Free Rotations for Post-Training Quantization. In *MLFor Systems workshop at Neural Information Processing Systems*.
 - [24] Alireza Ghaffari, Sharareh Younesian, Vahid Partovi Nia, Boxing Chen, and Masoud Asgharian. 2024. AdpQ: A Zero-shot Calibration Free Adaptive Post Training Quantization Method for LLMs. *arXiv preprint arXiv:2405.13358* (2024).
 - [25] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. *CoRR* (2021).
 - [26] Ruihao Gong, Yifu Ding, Zining Wang, Chengtao Lv, Xingyu Zheng, Jinyang Du, Yang Yong, Shiqiao Gu, Haotong Qin, Jinyang Guo, et al. 2025. A survey of low-bit large language models: Basics, systems, and algorithms. *Neural networks* (2025), 107856.

- [27] Ruihao Gong, Yang Yong, Shiqiao Gu, Yushi Huang, Chengtao Lv, Yunchen Zhang, Xianglong Liu, and Dacheng Tao. 2024. LLMC: Benchmarking Large Language Model Quantization with a Versatile Compression Toolkit. In *Findings of the Association for Computational Linguistics: EMNLP 2024 Industry Track*.
- [28] Hang Guo, Luca Benini, and Yawei Li. 2026. Optimal Brain Restoration for Joint Quantization and Sparsification of LLMs. In *The Fourteenth International Conference on Learning Representations*.
- [29] Liulu He, Shenli Zheng, Karwei Sun, Yijiang Liu, Yufei Zhao, Chongkang Tan, Huanrui Yang, Yuan Du, and Li Du. 2025. BASE-Q: Bias and Asymmetric Scaling Enhanced Rotational Quantization for Large Language Models. *arXiv preprint arXiv:2506.15689* (2025).
- [30] Xing Hu, Yuan Cheng, Dawei Yang, Zhixuan Chen, Zukang Xu, Jiangyong Yu, Chen Xu, Zhihang Yuan, Zhe Jiang, and Sifan Zhou. 2025. OSTQuant: Refining Large Language Model Quantization with Orthogonal and Scaling Transformations for Better Distribution Fitting. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*.
- [31] Haoyu Huang, Linlin Yang, Sheng Xu, Boyu Liu, Guodong Guo, Zhongqian Fu, Hang Zhou, and Baochang Zhang. 2026. FAIR-Calib: Frontier-Aware Instability-Rewighted Calibration for Post-Training Quantization of Diffusion Large Language Models. In *Forty-third International Conference on Machine Learning*.
- [32] Wei Huang, Yangdong Liu, Haotong Qin, Ying Li, Shiming Zhang, Xianglong Liu, Michele Magno, and Xiaojuan Qi. 2024. BiLLM: Pushing the Limit of Post-Training Quantization for LLMs. In *International Conference on Machine Learning*. PMLR, 20023–20042.
- [33] Wei Huang, Haotong Qin, Yangdong Liu, Yawei Li, Qinshuo Liu, Xianglong Liu, Luca Benini, Michele Magno, Shiming Zhang, and Xiaojuan Qi. 2025. SliM-LLM: Saliency-Driven Mixed-Precision Quantization for Large Language Models. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 (Proceedings of Machine Learning Research)*.
- [34] Minjun Kim, Jaehyeon Choi, Jongkeun Lee, Wonjin Cho, and U Kang. 2025. Zero-shot Quantization: A Comprehensive Survey. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence, IJCAI 2025, Survey Track*.
- [35] Suyoung Kim, Sunghyun Wee, Hyeonjin Kim, Kyomin Hwang, Hyunho Lee, and Nojun Kwak. 2026. ReSpinQuant: Efficient Layer-Wise LLM Quantization via Subspace Residual Rotation Approximation. In *Forty-third International Conference on Machine Learning*.
- [36] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. 2024. OWQ: Outlier-Aware Weight Quantization for Efficient Fine-Tuning and Inference of Large Language Models. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*. 13355–13364.
- [37] Janghwan Lee, Minsoo Kim, Seungcheol Baek, Seok Joong Hwang, Wonyong Sung, and Jungwook Choi. 2023. Enhancing Computation Efficiency in Large Language Models through Weight and Activation Quantization. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP*. 14726–14739.
- [38] Jemin Lee, Sihyeong Park, Jinse Kwon, Jihun Oh, and Yongin Kwon. 2025. Exploring the Trade-Offs: Quantization Methods, Task Difficulty, and Model Size in Large Language Models From Edge to Giant. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence, IJCAI 2025*.
- [39] Jung Hyun Lee, Jeonghoon Kim, Se Jung Kwon, and Dongsoo Lee. 2023. Flexround: Learnable rounding based on element-wise division for post-training quantization. In *International Conference on Machine Learning*.
- [40] Jung Hyun Lee, Jeonghoon Kim, June Yong Yang, Se Jung Kwon, Eunho Yang, Kang Min Yoo, and Dongsoo Lee. 2025. LRQ: Optimizing Post-Training Quantization for Large Language Models by Learning Low-Rank Weight-Scaling Matrices. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL - Volume 1*.
- [41] Shiyao Li, Yingchun Hu, Xuefei Ning, Xihui Liu, Ke Hong, Xiaotao Jia, Xiuhong Li, Yaqi Yan, Pei Ran, Guohao Dai, et al. 2025. MBQ: Modality-Balanced Quantization for Large Vision-Language Models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 4167–4177.
- [42] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. 2021. BRECO: Pushing the Limit of Post-Training Quantization by Block Reconstruction. In *9th International Conference on Learning Representations, ICLR*.
- [43] Yuhang Li and Priyadarshini Panda. 2024. Tesseraq: Ultra low-bit llm post-training quantization with block reconstruction. *arXiv preprint arXiv:2410.19103* (2024).
- [44] Yesheng Liang, Haisheng Chen, Zihan Zhang, Song Han, and Zhijian Liu. 2026. ParoQuant: Pairwise Rotation Quantization for Efficient Reasoning LLM Inference. In *The Fourteenth International Conference on Learning Representations (ICLR)*.
- [45] Haokun Lin, Haobo Xu, Yichen Wu, Jingzhi Cui, Yingtao Zhang, Linzhan Mou, Linqi Song, Zhenan Sun, and Ying Wei. 2024. DuQuant: Distributing Outliers via Dual Transformation Makes Stronger Quantized LLMs. In *The Thirty-eighth*

- Annual Conference on Neural Information Processing Systems*.
- [46] Haokun Lin, Haobo Xu, Yichen Wu, Ziyu Guo, Renrui Zhang, Zhichao Lu, Ying Wei, Qingfu Zhang, and Zhenan Sun. 2026. Quantization Meets dLLMs: A Systematic Study of Post-training Quantization for Diffusion LLMs. *Machine Intelligence Research* (2026).
- [47] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems*.
- [48] Han Liu, Haotian Gao, Xiaotong Zhang, Changya Li, Feng Zhang, Wei Wang, Fenglong Ma, and Hong Yu. 2025. Septq: A simple and effective post-training quantization paradigm for large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 812–823.
- [49] Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng YU, Chun Yuan, and Lu Hou. 2025. Quantization Hurts Reasoning? An Empirical Study on Quantized Reasoning Models. In *Second Conference on Language Modeling*.
- [50] Yijun Liu, Yuan Meng, Fang Wu, Shenhao Peng, Hang Yao, Chaoyu Guan, Chen Tang, Xinzhua Ma, Zhi Wang, and Wenwu Zhu. 2024. Evaluating the generalization ability of quantized llms: Benchmark, analysis, and toolbox. *arXiv preprint arXiv:2406.12928* (2024).
- [51] Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye, Li Lina Zhang, Ting Cao, Cheng Li, and Mao Yang. 2024. VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*. 8181–8196.
- [52] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen (Henry) Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *Forty-first International Conference on Machine Learning, ICML*.
- [53] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. 2025. SpinQuant: LLM Quantization with Learned Rotations. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*.
- [54] Sanae Lotfi, Polina Kirichenko, Steven Li, and Zechun Liu. 2026. Quantized Reasoning Models Think They Need to Think Longer, but They Do Not. *arXiv preprint arXiv:2606.00206* (2026).
- [55] Gaurav Menghani. 2023. Efficient Deep Learning: A Survey on Making Deep Learning Models Smaller, Faster, and Better. *ACM Comput. Surv.* 55, 12, Article 259 (March 2023), 37 pages.
- [56] Meta Platforms, Inc. 2024. Llama 3: Model Implementation. <https://github.com/meta-llama/llama3/blob/main/llama/model.py>. Accessed: 2026-08-24.
- [57] Meta Platforms, Inc. 2026. Llama Models: Model SKU List. https://github.com/meta-llama/llama-models/blob/main/models/sku_list.py. Accessed: 2026-08-24.
- [58] Mohammad Mozaffari, Amir Yazdanbakhsh, and Maryam Mehri Dehnavi. 2025. SLiM: One-shot Quantization and Sparsity with Low-rank Approximation for LLM Weight Compression. In *Proceedings of the 42nd International Conference on Machine Learning, ICML*.
- [59] Lorenz K Muller, Philippe Bich, Jiawei Zhuang, Ahmet Çelik, Luca Benfenati, and Lukas Cavigelli. 2026. SINQ: Sinkhorn-Normalized Quantization for Calibration-Free Low-Precision LLM Weights. In *Forty-third International Conference on Machine Learning*.
- [60] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos, and Tijmen Blankevoort. 2020. Up or Down? Adaptive Rounding for Post-Training Quantization. In *Proceedings of the 37th International Conference on Machine Learning, ICML*.
- [61] Piotr Nawrot, Jianing Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo Maria Ponti. 2026. The sparse frontier: Sparse attention trade-offs in transformer llms. In *Findings of the Association for Computational Linguistics: ACL 2026*. 38667–38701.
- [62] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, JUN ZHOU, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large Language Diffusion Models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [63] NVIDIA. 2025. TensorRT-LLM: Optimizing Large Language Model Inference. Accessed: 2025-12-30.
- [64] NVIDIA, Felix Abecassis, Anjulie Agrusa, Dong Ahn, Jonah Alben, Stefania Alborghetti, Michael Andersch, Sivakumar Arayandi, Alexis Bjorlin, Aaron Blakeman, et al. 2025. Pretraining Large Language Models with NVFP4. *arXiv preprint arXiv:2509.25149* (2025).
- [65] Sangki Park and Ki-Seok Chung. 2026. SpinOut: Enhanced Rotation-Based Quantization for LLM by Outlier Injection. *IEEE Access* 14 (2026), 24082–24095.
- [66] Alessandro Pierro and Steven Abreu. 2024. Mamba-PTQ: Outlier Channels in Recurrent Large Language Models. Efficient Systems for Foundation Models Workshop, ICML 2024.

- [67] Shahzeb Qamar, Lorenz Sparrenberg, Christian Bauckhage, Baha Rababah, Carson Leung, Murat Kantarcioglu, Cuneyt Gurcan Akcora, and Rafet Sifa. 2026. Accuracy is Not Enough: A Divergence-Based Approach to Evaluate Fidelity Loss in Quantized LLMs. In *2026 IEEE international conference on data science and advanced analytics (DSAA)*. IEEE.
- [68] Guangshuo Qin, Zhiteng Li, Zheng Chen, Weihang Zhang, Linghe Kong, and Yulun Zhang. 2026. VEQ: Modality-Adaptive Quantization for MoE Vision-Language Models. In *Forty-third International Conference on Machine Learning*.
- [69] Baha Rababah, Shahzeb Qamar, Lorenz Sparrenberg, Rafet Sifa, Murat Kantarcioglu, Cuneyt Gurcan Akcora, and Carson K Leung. 2026. The Illusion of Equivalency: Statistical Characterization of Quantization Effects in LLMs. *arXiv preprint arXiv:2607.08734* (2026).
- [70] Sai Sanjeet, Ian Colbert, Pablo Monteagudo-Lago, Giuseppe Franco, Yaman Umuroglu, and Nicholas J. Fraser. 2026. Pushing the Limits of Block Rotations in Post-Training Quantization. *arXiv preprint arXiv:2601.22347* (2026).
- [71] Utkarsh Saxena and Kaushik Roy. 2025. KVLinC: KV Cache Quantization with Hadamard Rotation and Linear Correction. *arXiv preprint arXiv:2510.05373* (2025).
- [72] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2024. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. In *The Twelfth International Conference on Learning Representations, ICLR*.
- [73] Yuantian Shao, Yuanteng Chen, Peisong Wang, Jianlin Yu, Jing Lin, yiwu yao, Zhihui Wei, and Jian Cheng. 2025. DartQuant: Efficient Rotational Distribution Calibration for LLM Quantization. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 143936–143970. doi:10.52202/085713-4819
- [74] Yuantian Shao, Peisong Wang, Yuanteng Chen, Chang Xu, Zhihui Wei, and Jian Cheng. 2026. Block Rotation is All You Need for MXFP4 Quantization. In *Forty-third International Conference on Machine Learning*.
- [75] Tianyao Shi and Yi Ding. 2025. Systematic characterization of llm quantization: A performance, energy, and quality perspective. *arXiv preprint arXiv:2508.16712* (2025).
- [76] Zunhai Su, Zhe Chen, Wang Shen, Hanyu Wei, Ling Li, Huangqi Yu, and Kehong Yuan. 2025. RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for LLMs via Outlier-Aware Adaptive Rotations. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence (IJCAI)*.
- [77] Zunhai Su and Kehong Yuan. 2025. KVSink: Understanding and Enhancing the Preservation of Attention Sinks in KV Cache Quantization for LLMs. In *Second Conference on Language Modeling*.
- [78] Yuxuan Sun, Ruikang Liu, HaoLi Bai, Han Bao, Kang Zhao, Yuening Li, Jiaxin Hu, Xianzhi Yu, Lu Hou, Chun Yuan, Xin Jiang, Wulong Liu, and Jun Yao. 2025. FlatQuant: Flatness Matters for LLM Quantization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. arXiv:2410.09426.
- [79] Hanlin Tang, Yifu Sun, Decheng Wu, Kai Liu, Jianchen Zhu, and Zhanhui Kang. 2023. EasyQuant: An Efficient Data-free Quantization Algorithm for LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP*.
- [80] Wei Tao, Bin Zhang, Xiaoyang Qu, Jiguang Wan, and Jianzong Wang. 2025. Cocktail: Chunk-Adaptive Mixed-Precision Quantization for Long-Context LLM Inference. In *Design, Automation and Test in Europe Conference, DATE*.
- [81] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*.
- [82] turboderp. 2025. exllama: A Memory-Efficient LLaMA Implementation for Quantized Weights. Accessed: 2025-12-30.
- [83] Mart Van Baalen, Andrey Kuzmin, Ivan Koryakovskiy, Markus Nagel, Peter Couperus, Cedric Bastoul, Eric Mahurin, Tijmen Blankevoort, and Paul Whatmough. 2024. Gptvq: The blessing of dimensionality for llm quantization. *arXiv preprint arXiv:2402.15319* (2024).
- [84] Changyuan Wang, Ziwei Wang, Xiuwei Xu, Yansong Tang, Jie Zhou, and Jiwen Lu. 2024. Q-VLM: Post-training Quantization for Large Vision-Language Models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [85] Qianli Wang, Mingyang Wang, Nils Feldhus, Simon Ostermann, Yuan Cao, Hinrich Schütze, Sebastian Möller, and Vera Schmitt. 2026. Through a Compressed Lens: Investigating The Impact of Quantization on Factual Knowledge Recall. In *Proceedings of the TrustNLP Workshop at ACL 2026*.
- [86] Sunghyun Wee, Suyoung Kim, Hyeonjin Kim, Kyomin Hwang, and Nojun Kwak. 2025. Safety-Preserving PTQ via Contrastive Alignment Loss. *arXiv preprint arXiv:2511.07842* (2025).
- [87] Mingrun Wei, Yeyu Yan, and Dong Wang. 2025. MPPQ: enhancing post-training quantization for LLMs via mixed supervision, proxy rounding, and pre-searching. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. 8277–8285.
- [88] Miles Williams, George Chrysostomou, and Nikolaos Aletras. 2025. Self-calibration for Language Model Quantization and Pruning. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025, Volume 1: Long Papers*. Association for

- Computational Linguistics.
- [89] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *International Conference on Machine Learning, ICML 2023*.
 - [90] Jinying Xiao, Bin Ji, Shasha Li, Xiaodong Liu, Ma Jun, Chao Wang, Wei Li, Ye Zhong, Xuan Xie, Nyima Tashi, et al. 2025. Outlier Smoothing with Closed-Form Rotations for W4A4 Large Language Model Quantization. *arXiv preprint arXiv:2511.22316* (2025).
 - [91] Bingxin Xu, Zhen Dong, Oussama Elachqar, and Yuzhang Shang. 2025. Butterflyquant: Ultra-low-bit llm quantization through learnable orthogonal butterfly transforms. *arXiv preprint arXiv:2509.09679* (2025).
 - [92] Bruce Changlong Xu, Adarsh Kumarappan, and Mu Zhou. 2026. Alignment Collapse Under KV Cache Quantization: A 35-Minute Audit for Quantized LLM Deployments. In *AdaptFM: Resource-Adaptive Foundation Model Inference wORKS, ICML*.
 - [93] Yuzhuang Xu, Shiyu Ji, Qingfu Zhu, and Wanxiang Che. 2025. CRVQ: Channel-Relaxed Vector Quantization for Extreme Compression of LLMs. *Transactions of the Association for Computational Linguistics* 13 (2025), 1488–1506.
 - [94] Zukang Xu, Xing Hu, Qiang Wu, and Dawei Yang. 2025. RSAVQ: Riemannian Sensitivity-Aware Vector Quantization for Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 38.
 - [95] Xianglong Yan, Chengzhu Bao, Zhiteng Li, Tianao Zhang, Kaicheng Yang, Haotong Qin, Ruobing Xie, Xingwu Sun, and Yulun Zhang. 2026. PT²-LLM: Post-Training Ternarization for Large Language Models. In *The Fourteenth International Conference on Learning Representations, ICLR*.
 - [96] Haiqi Yang, Zhiyuan Li, Yi Chang, and Yuan Wu. 2026. A Survey of Retentive Network. In *Findings of the Association for Computational Linguistics: ACL 2026*. 5200–5216.
 - [97] Jiawei Yang, Zhongbo Li, Zeqin Feng, and Yongqiang Xie. 2025. A survey on neural network quantization. In *Proceedings of the 2025 6th International Conference on Computer Information and Big Data Applications*. 384–394.
 - [98] Ling Yang, Ye Tian, Bowen Li, Xincheng Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. 2025. MMaDA: Multimodal Large Diffusion Language Models. In *Advances in Neural Information Processing Systems*, Vol. 38. 138867–138907.
 - [99] Junho Yoon, Geom Lee, Donghyeon Jeon, Inho Kang, and Seung-Hoon Na. 2025. ROSAQ: Rotation-based Saliency-Aware Weight Quantization for Efficiently Compressing Large Language Models. *arXiv preprint arXiv:2506.13472* (2025).
 - [100] Zhihang Yuan, Yuzhang Shang, and Zhen Dong. 2024. PB-LLM: Partially Binarized Large Language Models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*.
 - [101] Cheng Zhang, Jianyi Cheng, George A Constantinides, and Yiren Zhao. 2024. LQER: low-rank quantization error reconstruction for LLMs. In *Proceedings of the 41st International Conference on Machine Learning*. 58763–58779.
 - [102] Cheng Zhang, Jeffrey TH Wong, Can Xiao, George Constantinides, and Yiren Zhao. 2025. Qera: an analytical framework for quantization error reconstruction. In *International Conference on Learning Representations*.
 - [103] Tianao Zhang, Zhiteng Li, Xianglong Yan, Haotong Qin, Yong Guo, and Yulun Zhang. 2026. Quant-dLLM: Post-Training Extreme Low-Bit Quantization for Diffusion Large Language Models. In *International Conference on Learning Representations*.
 - [104] Jiaqi Zhao, Ming Wang, Miao Zhang, Yuzhang Shang, Xuebo Liu, Yaowei Wang, Min Zhang, and Liqiang Nie. 2025. Benchmarking post-training quantization in llms: Comprehensive taxonomy, unified evaluation, and comparative analysis. *arXiv preprint arXiv:2502.13178* (2025).
 - [105] Jiaqi Zhao, Miao Zhang, Ming Wang, Yuzhang Shang, Kaihao Zhang, Weili Guan, Yaowei Wang, and Min Zhang. 2025. Ptq1. 61: Push the real limit of extremely low-bit post-training quantization methods for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4483–4502.
 - [106] Jiaqi Zhao, Miao Zhang, Chao Zeng, Ming Wang, Xuebo Liu, and Liqiang Nie. 2024. Lrquant: Learnable and robust post-training quantization for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2240–2255.
 - [107] Weibo Zhao, Yubin Shi, Xinyu Lyu, Wanchen Sui, Shen Li, and Yong Li. 2025. ASER: activation smoothing and error reconstruction for large language model quantization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
 - [108] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2024. A Survey on Model Compression for Large Language Models. *Trans. Assoc. Comput. Linguistics* 12 (2024), 1556–1577.

Received 20 August 2026